AegisRL

Adaptive Reinforcement Learning for Hospital Electronic Health Record Access Control

● 300+ Questions ● 36 Sections ● Code-Level Deep Dives
● Reward Tables ● Numerical Calculations ● Professor Follow-Up Chains
● Rapid-Fire Round ● Final Self-Test ● Debugging Scenarios

Component	Detail
Algorithm	Tabular Q-Learning (Off-policy TD Control)
State Space	1,152 states (4×2×4×2×3×3×2)
Action Space	4 actions: ALLOW, DENY, CHALLENGE, RESTRICT
Q-Table Shape	(1152, 4) → 4,608 values · Stored: models/q_table.json
Training	35,000 episodes · α=0.10 · γ=0.90 · ε: 1.0→0.01 · seed=42
Evaluation	5,000 requests · seed=100
Attack Blocking Rate	99.37%
Legitimate Access Rate	99.79%
False Denial Rate	0.21%
Emergency Denial Rate	0.13%
Decision Latency	0.0184 ms (mean) · 0.0222 ms (p95)
Cumulative Reward	+71,363.0
Backend	FastAPI + Uvicorn (ASGI) · Python 3.14
Database	Supabase PostgreSQL (7 tables)
Frontend	Vanilla JS/CSS/HTML SPA · 6 dashboards

■ IMPORTANT IMPLEMENTATION NOTE – Read Before Your Viva

The agent.py constructor default is epsilon_decay=0.9997, but the train_agent() function in train.py explicitly passes epsilon_decay=0.9998. The dossier and all reported results use 0.9998. If a professor looks at agent.py and asks 'why 0.9997?', clarify that train.py overrides the default with 0.9998.

Table of Contents

Section 1: Basic Project Questions
Section 2: Tell Me About Your Project
Section 3: Libraries & Technology Stack
Section 4: File Structure & Code
Section 5: State Representation
Section 6: State Indexing (Deep Dive)
Section 7: Action Space
Section 8: Reward Function
Section 9: MDP Theory
Section 10: Q-Learning Theory
Section 11: Epsilon-Greedy
Section 12: Training
Section 13: Training Results & Convergence
Section 14: Attack Simulation
Section 15: Database
Section 16: FastAPI / Backend
Section 17: Frontend
Section 18: Database + RL Integration
Section 19: Evaluation
Section 20: Baseline Comparison
Section 21: Code-Level 'Psycho Sir' Questions
Section 22: Why Did You Do It This Way?
Section 23: What If I Change This? (Counterfactuals)
Section 24: RL Theory Beyond the Project
Section 25: Connecting Theory to Your Project
Section 26: Trick Questions
Section 27: Numerical / Calculation Questions
Section 28: Debugging Questions
Section 29: Security Questions
Section 30: Limitations
Section 31: Future Work
Section 32: Professor Rapid-Fire Round (Psycho Sir)
Section 33: Follow-Up Chains
Section 34: Code Walkthrough Viva
Section 35: Show Me On The Project

Section 36: Final Viva Self-Test

Section 1: Absolute Basic Questions

These are the questions a professor will ask in the very first minute. Nail these before anything else.

Q1. What is your project?

[Easy]

Viva Answer: My project is AegisRL – Adaptive Reinforcement Learning for Hospital EHR Access Control. Instead of using a fixed rule-based system to decide who can access patient records, I trained a Q-Learning agent that looks at seven contextual features of each access request and dynamically decides whether to ALLOW, DENY, CHALLENGE, or RESTRICT access.

Q2. What does AegisRL stand for?

[Easy]

Viva Answer: 'Aegis' is a Greek word meaning a protective shield. It's the shield of Zeus in mythology. I chose it because the project acts as an intelligent shield that protects patient records from unauthorized access. 'RL' stands for Reinforcement Learning.

Q3. What problem are you solving?

[Easy]

Viva Answer: Hospitals use static Role-Based Access Control – RBAC – which basically just checks your job title. That creates two problems: first, in emergencies, it blocks doctors from accessing records they urgently need because they're from a different department. Second, if credentials are stolen, the attacker gets in because static RBAC only checks who you claim to be, not how, when, or where the request is coming from. I'm solving that by adding contextual intelligence.

Q4. Why is this a Reinforcement Learning problem?

[Easy]

Viva Answer: Because we need an agent that makes sequential decisions and learns from feedback. There's no labeled training dataset saying 'this request should be CHALLENGED'. Instead, the agent explores different actions and receives rewards – positive for good decisions, negative for bad ones – and gradually learns the optimal policy. That trial-and-reward structure is exactly what RL is designed for.

If Sir Pushes Further: *Why not just use a rule engine? Rules don't adapt. If an attacker learns the rules, they bypass them. The RL agent's policy emerged from the reward function, so it captures nuanced trade-offs that are hard to hand-code.*

Q5. Why not use supervised learning?

[Easy]

Viva Answer: Supervised learning needs labeled data – you need examples saying 'this request was an attack' or 'this was legitimate'. In a new hospital, you don't have thousands of labeled attack incidents. RL doesn't need historical labels; it learns by interacting with a simulated environment and receiving reward signals designed by the system engineers.

Q6. Why not use a simple classification model?

[Easy]

Viva Answer: Two reasons. First, classification gives you binary labels – attack or not. We need four graded actions: ALLOW, DENY, CHALLENGE, RESTRICT. That's a multi-action decision problem, not a binary one. Second, the optimal action depends on the entire context – role, location, frequency, emergency flag – in a way that's hard to capture with a single classifier without heavy feature engineering.

Q7. Why not use a rule-based system?

[Easy]

Viva Answer: Rule-based systems are brittle. You'd need to manually write rules like 'if external and off-hours and high-sensitivity then DENY', and as soon as the threat pattern changes, you have to update the rules manually. The RL agent learns the right response automatically from the reward function, including nuanced situations like an internal employee snooping during working hours, which is hard to express as a simple rule.

Q8. What is the agent in your project?

[Easy]

Viva Answer: The agent is the TabularQLearningAgent class defined in agent.py. It maintains a Q-table of shape 1152×4 and uses it to decide which access control action to take for each incoming request.

Code Connection: *agent.py → class TabularQLearningAgent, line 29*

Q9. What is the environment?

[Easy]

Viva Answer: The environment is the AccessRequestEnvironment class in env.py. It generates synthetic hospital access requests according to one of five scenario distributions, and it calculates the reward for each action the agent takes. It simulates what would happen if a hospital's access control system received real requests.

Code Connection: *env.py → class AccessRequestEnvironment, line 65*

Q10. What is a state?

[Easy]

Viva Answer: A state is a 7-tuple that describes the context of an access request: role, department match, record sensitivity, request time, network location, request frequency, and emergency flag. Together they uniquely describe the situation the agent is in, and together they form one of 1,152 possible discrete states.

Code Connection: *env.py → class Observation (NamedTuple), line 55*

Q11. What is an action?

[Easy]

Viva Answer: An action is the access control decision the agent makes. There are four: ALLOW means full access, DENY means block, CHALLENGE means ask for step-up verification like MFA, and RESTRICT means give partial or masked access. Each is encoded as an integer: 0, 1, 2, 3.

Code Connection: *config.py → ACTION_ALLOW=0, ACTION_DENY=1, ACTION_CHALLENGE=2, ACTION_RESTRICT=3*

Q12. What is a reward?

[Easy]

Viva Answer: A reward is a numeric score the environment gives to the agent after it takes an action. Positive rewards reinforce good decisions – like ALLOW during an emergency (+20) – and negative rewards penalize bad ones – like DENY during an emergency (-35). The agent learns to maximize total reward over time.

Q13. What is a policy?

[Easy]

Viva Answer: A policy is the mapping from states to actions – it tells the agent what to do in each situation. During inference, my policy is deterministic: always pick the action with the highest Q-value in that state. During training, I use an epsilon-greedy policy that sometimes picks randomly to explore.

Q14. What is a Q-value?

[Easy]

Viva Answer: A Q-value, Q(s,a), is the estimated total cumulative reward the agent expects to get if it takes action a in state s and follows the optimal policy from then on. A higher Q-value means that action is better in that context. The agent selects the action with the highest Q-value.

Q15. What is an episode in your project?

[Easy]

Viva Answer: Each access request is one episode. The agent receives the observation, picks an action, gets a reward, and that's it – done=True. There's no multi-step trajectory because each access decision is independent. So one episode = one access request = one Q-table update.

Code Connection: *env.py → step() function, line 252: done = True*

Q16. What is exploration vs exploitation?

[Easy]

Viva Answer: Exploration means the agent tries random actions to discover new information – even if they might be bad. Exploitation means using what it already knows – picking the best-known action. We need both: too much exploitation early on means the agent never discovers better options; too much exploration late on means it never uses what it learned.

Q17. What is the objective of the agent?

[Easy]

Viva Answer: To maximize the expected cumulative discounted reward over all episodes. Concretely, this means the agent should learn to ALLOW legitimate clinical access, DENY or CHALLENGE attacks, and handle emergencies without denying critical care access.

Q18. What is a trajectory?

[Easy]

Viva Answer: In general RL, a trajectory is the sequence of states, actions, and rewards across an entire episode. In my project, because each episode is single-step, the trajectory is simply (s, a, r) – one state, one action, one reward.

Section 2: Tell Me About Your Project

Q19. Explain your project in 30 seconds.

[Easy]

Viva Answer: AegisRL trains a Q-Learning agent to make access control decisions for hospital EHR systems. It observes 7 features about each request and picks from 4 actions: ALLOW, DENY, CHALLENGE, or RESTRICT. It achieves 99.37% attack blocking and 99.79% legitimate access with sub-millisecond latency.

Q20. Explain the complete workflow from frontend to decision.

[Medium]

Viva Answer: A hospital staff member fills out a request in the web dashboard. The frontend sends an HTTP POST to /predict in FastAPI. FastAPI calls parse_context_to_observation() to convert the request fields into a discrete 7-tuple, then state_to_index() converts it to a number from 0 to 1151. The agent looks up Q[state_idx] and picks the action with the highest Q-value using argmax. The decision, Q-values, and explanation are returned as JSON and also logged to three Supabase tables: access_requests, rl_decisions, and audit_logs. The frontend renders the decision badge.

Code Connection: *main.py → evaluate_access_request(), lines 207-266*

Q21. What happens inside /predict?

[Medium]

Viva Answer: First, timing starts with time.perf_counter(). Then parse_context_to_observation() maps the JSON body to a discrete Observation namedtuple. state_to_index() converts that to an integer. agent.choose_action(state_idx, training=False) does the greedy argmax – no random action during inference. Then get_q_values() and explain_decision() are called. Finally log_access_and_decision() tries to write to Supabase. The whole thing is returned as a DecisionResponseSchema JSON object.

Code Connection: *main.py → evaluate_access_request(), line 207*

Q22. How does a request become a state index?

[Medium]

Viva Answer: parse_context_to_observation() maps each field to its discrete integer – role string becomes 0/1/2/3, sensitivity string becomes 0/1/2/3, and so on. That gives us an Observation namedtuple. Then state_to_index() applies the weighted sum formula: role×288 + dept_match×144 + sensitivity×36 + time×18 + location×6 + frequency×2 + is_emergency. The result is a unique index from 0 to 1151.

Code Connection: *env.py → state_to_index(), line 82; env.py → parse_context_to_observation(), line 406*

Q23. Where is the Q-table stored and how is it loaded?

[Medium]

Viva Answer: The Q-table is stored as a JSON file at models/q_table.json. It contains the num_states, num_actions, actions list, and the q_table as a nested list. On FastAPI startup, the load() method in TabularQLearningAgent reads this file, converts the nested list to a NumPy array

with np.array(..., dtype=np.float64), and sets self.q_table. This happens once at startup; no retraining occurs per API call.

Code Connection: *agent.py → load(), line 185; main.py → startup_event(), line 69*

Q24. Is training performed during every API request?

[Medium]

Viva Answer: No. Training is completely offline. train.py runs once to produce the Q-table JSON file. The API only reads the Q-table and uses it for inference. The Q-table values never change after the API starts. This is a deliberate design decision for deterministic, fast inference.

Q25. Why did you separate training and inference?

[Medium]

Viva Answer: For safety and predictability. In a hospital system, you don't want the access-control model changing its behavior mid-operation due to live updates. Offline training means the model is validated before deployment. It also makes inference very fast – just a memory lookup.

Q26. How is the decision logged?

[Medium]

Viva Answer: After inference, log_access_and_decision() in db.py is called. It inserts a row into access_requests with user/patient/record IDs, location, emergency status, and department match. It gets the new request_id back, then inserts into rl_decisions linking to that request_id with the action, reward proxy, and latency. Finally it inserts into audit_logs with a human-readable event description.

Code Connection: *db.py → log_access_and_decision(), line 337*

Section 3: Libraries & Technology Stack

Library/Tool	Version	Purpose	Where Used
NumPy	≥2.0.0	Q-table (np.zeros), argmax, percentile calculations	agent.py, evaluate.py
FastAPI	≥0.115.0	REST API framework, Pydantic schemas, endpoint routing	main.py
Uvicorn	≥0.30.0	ASGI server that runs FastAPI app	CLI startup
Pydantic	≥2.8.0	Request/Response schema validation	main.py
Supabase-py	≥2.8.0	PostgreSQL client via PostgREST API	db.py
Plotly	≥5.24.0	Training convergence charts, evaluation bar charts	train.py, evaluate.py
python-dotenv	≥1.0.0	Load .env credentials for Supabase	config.py
requests	≥2.32.0	OpenAPI schema introspection HTTP call	db.py
httpx	≥0.27.0	HTTP testing / async HTTP	test files
ReportLab	≥4.2.5	PDF generation for reports	generate_dossier_pdf.py

Q27. Why did you use FastAPI instead of Flask or Django?

[Medium]

Viva Answer: FastAPI is async-native and automatically generates OpenAPI documentation. It's much faster than Flask for concurrent requests because it's built on Starlette and ASGI. It also has built-in Pydantic validation so I get automatic type-checking on request bodies without writing extra code. Django would be overkill for a REST-only backend.

Q28. What is ASGI and how does it relate to Uvicorn?

[Medium]

Viva Answer: ASGI stands for Asynchronous Server Gateway Interface. It's the modern Python standard for async web servers. FastAPI is an ASGI application. Uvicorn is the ASGI server that actually listens on the port and forwards requests to FastAPI. Uvicorn runs on top of uvloop for high performance. Without Uvicorn, FastAPI has no server to run on.

Q29. What is NumPy doing in your project exactly?

[Medium]

Viva Answer: Three things. First, it stores the Q-table as a float64 2D array of shape (1152, 4). Second, np.random.rand() and np.random.randint() drive the epsilon-greedy random action selection. Third, np.max() finds the best next-state Q-value, np.where() and np.isclose() handle tie-breaking in argmax. Also np.mean() and np.percentile() compute evaluation metrics.

Code Connection: *agent.py lines 53, 62-68, 88; evaluate.py lines 132-133*

Q30. Why is the Q-table stored as JSON and not a binary format like .npy?

[Medium]

Viva Answer: JSON is human-readable and easy to inspect manually or load in any language or frontend. For 1152×4 = 4608 float values, the JSON file is about 81 KB, which is tiny. There's no performance disadvantage at this scale. It also makes the model artifact easy to share and audit.

Code Connection: *agent.py → save(), line 150; models/q_table.json*

Q31. Why Supabase instead of a local PostgreSQL database?

[Medium]

Viva Answer: Supabase provides a hosted PostgreSQL instance with a Python SDK, REST API via PostgREST, and authentication built in. For a prototype, it removes the need to manage a local database server. The actual storage engine is still PostgreSQL 15, so all relational guarantees apply.

Q32. Why Plotly for visualization?

[Easy]

Viva Answer: Plotly generates interactive HTML charts with zoom, hover tooltips, and legends out of the box. The training convergence chart and evaluation bar chart need to be inspected by the researcher – interactive charts are far more useful than static matplotlib images for that purpose.

Code Connection: *train.py → generate_plotly_training_chart(), line 132; evaluate.py → generate_plotly_eval_chart(), line 197*

Q33. What is python-dotenv doing?

[Easy]

Viva Answer: It reads the SUPABASE_URL and SUPABASE_KEY from the .env file into environment variables so they're never hardcoded in the source. config.py calls load_dotenv() and then os.getenv() to retrieve them. This keeps credentials out of the Git repository.

Code Connection: *config.py → load_dotenv(), line 14*

Q34. What does the requests library do in db.py?

[Medium]

Viva Answer: It makes an HTTP GET to the Supabase PostgREST OpenAPI endpoint to retrieve the table schema definitions. This is the Schema 'Strategy 1' in inspect_schema(). It lets db.py discover what columns actually exist in each table without assuming column names.

Code Connection: *db.py → inspect_schema(), lines 68-79*

Q35. What does CORS middleware do in your FastAPI app?

[Medium]

Viva Answer: CORS (Cross-Origin Resource Sharing) middleware allows the browser-based frontend to call the API from a different origin. Without it, the browser would block the JavaScript fetch() calls. The current configuration allows all origins ('*'), which is fine for a development prototype but should be restricted to specific domains in production.

Code Connection: *main.py → CORSMiddleware, lines 43-49*

Section 4: Project File Structure & Code

File	Responsibility
config.py	All constants: action names, role/sensitivity/location maps, file paths, Supabase credentials
env.py	AccessRequestEnvironment class: scenario generation, state_to_index, index_to_state, calculate_rewar
agent.py	TabularQLearningAgent class: Q-table init, choose_action, update, decay_epsilon, save, load, explain_d
train.py	train_agent() function: 35,000-episode training loop, Plotly chart generation, model save
evaluate.py	evaluate_agent() function: 5,000-request evaluation, all KPI metrics, scenario breakdown matrix
main.py	FastAPI app: all endpoints (/predict, /simulate, /health, /metrics, etc.), startup model load
db.py	Supabase client, schema inspection, payload adaptation, log_access_and_decision(), audit trail
models/q_table.json	Trained Q-table: num_states=1152, num_actions=4, 4608 float64 values
models/agent_metadata.json	Hyperparameter snapshot: alpha, gamma, epsilon, decay, episodes trained
models/training_metrics.json	Training checkpoint data: rewards, epsilon, TD errors at 1000-episode intervals
static/index.html	Single Page Application frontend: 6 dashboard views
static/js/access.js	RL decision engine UI: fetch /predict, render Q-value bars, animate pipeline
static/js/simulator.js	HTML5 Canvas cyber attack simulator: network packet animations
static/js/app.js	Navigation, theme toggling, API info calls
static/js/dashboard.js	KPI cards, stats fetch from /api/stats

Q36. What is config.py responsible for?

[Easy]

Viva Answer: It's the single source of truth for all constants. Action names, action integer codes, role/sensitivity/ location/frequency string-to-integer maps, file paths for the model and metadata, Supabase credentials loaded from .env, and the list of required database tables. Any file that needs these values imports from config.py instead of redefining them.

Code Connection: *config.py, all lines*

Q37. What are the important functions in env.py?

[Medium]

Viva Answer: state_to_index() converts a 7-tuple observation to a unique integer 0–1151. index_to_state() does the reverse. generate_request() samples a scenario and produces both the observable Observation namedtuple and the hidden_meta dict. calculate_reward() applies the piecewise reward matrix given the scenario. parse_context_to_observation() converts incoming API JSON into a discrete Observation.

Code Connection: *env.py lines 82, 97, 118, 266, 406*

Q38. What are the important functions in agent.py?

[Medium]

Viva Answer: choose_action() does epsilon-greedy during training and greedy argmax during inference, with np.isclose tie-breaking. update() applies the Bellman TD update rule. decay_epsilon() multiplies epsilon by epsilon_decay after each episode and clamps it at epsilon_min. save() serializes the Q-table to JSON. load() deserializes it back and verifies the shape. explain_decision() generates the natural language rationale.

Code Connection: *agent.py lines 57, 71, 95, 150, 185, 109*

Q39. How does choose_action() handle ties between equal Q-values?

[Hard]

Viva Answer: It uses np.isclose() to find all actions whose Q-value is within floating-point tolerance of the maximum, then calls np.random.choice() on that set to pick uniformly at random. This prevents a systematic bias toward the first action in case of ties, which could happen if the Q-table has many unvisited states still at zero.

Code Connection: *agent.py → choose_action(), lines 66-69*

Q40. What does train.py's train_agent() do step by step?

[Medium]

Viva Answer: It creates the environment with seed=42 and the agent. Then for 35,000 episodes: calls env.reset() to get an observation, state_to_index() to get the state index, choose_action() in training mode, env.step() to get next state/reward/done/info, agent.update() for the Q-table update, and decay_epsilon(). Every 1,000 episodes it logs the average reward, epsilon, and TD error. After training it saves the model and generates a Plotly HTML chart.

Code Connection: *train.py → train_agent(), lines 24-129*

Q41. What does evaluate.py measure and how?

[Hard]

Viva Answer: It loads the trained agent, creates a fresh environment with seed=100, then loops 5,000 times. Each iteration calls generate_request() to get obs and hidden_meta, state_to_index(), then choose_action() with training=False. It categorizes the result: if not legitimate and action is DENY or CHALLENGE (or RESTRICT for insider), it counts as attack_blocked; if not legitimate and ALLOW, it's unauthorized. For legitimate requests, DENY counts as false_denial. Latency is measured with time.perf_counter() around the argmax call only.

Code Connection: *evaluate.py → evaluate_agent(), lines 30-194*

Q42. What does db.py's adapt_payload_to_schema() do?

[Hard]

Viva Answer: Before inserting a record, it fetches the actual column names from the cached schema and filters the payload to only include keys that match those columns. It also handles case differences and common synonyms (e.g. 'action' vs 'decision', 'created_at' vs 'timestamp'). This prevents PostgREST column-not-found errors without needing to hardcode column names.

Code Connection: *db.py → adapt_payload_to_schema(), lines 165-219*

Section 5: State Representation (Deep Dive)

Feature	Values	Count	Encoding
Role	DOCTOR, NURSE, STAFF, RESEARCHER	4	0, 1, 2, 3
Dept Match	MISMATCH, MATCH	2	0, 1
Record Sensitivity	LOW, MEDIUM, HIGH, CRITICAL_VIP	4	0, 1, 2, 3
Request Time	NORMAL_HOURS, OFF_HOURS	2	0, 1
Location	INTERNAL_HOSPITAL, VPN_OFFSITE, EXTERNAL_UNKNOWN	3	0, 1, 2
Request Frequency	NORMAL(≤5/min), ELEVATED(6-20/min), BURST_HIGH(>20/min)	3	0, 1, 2
Emergency Flag	NO, YES	2	0, 1

Q43. How did you get 1,152 states?

[Easy]

Viva Answer: Just multiply the cardinalities of all seven features: 4 × 2 × 4 × 2 × 3 × 3 × 2 = 1,152. Let me show the steps: 4×2 = 8, ×4 = 32, ×2 = 64, ×3 = 192, ×3 = 576, ×2 = 1,152.

Quick Calculation: *4 × 2 = 8 → × 4 = 32 → × 2 = 64 → × 3 = 192 → × 3 = 576 → × 2 = 1,152*

Q44. Why these seven specific features?

[Medium]

Viva Answer: Each feature addresses a known attack vector or clinical safety need. Role tells us what level of access is expected. Department match catches cross-unit snooping. Sensitivity identifies high-value targets. Request time flags off-hours anomalies. Location distinguishes internal hospital access from external attacks. Frequency detects automated scraping. Emergency flag prevents denying critical care access. Together they cover the ABAC factors documented in the literature.

Q45. Why is Role included as a feature?

[Easy]

Viva Answer: Different roles have different baseline access expectations. A Doctor has broader clinical access than a Staff member. Role lets the agent calibrate its response – ALLOW is less suspicious for a Doctor accessing a patient record than for a Researcher accessing the same record.

Q46. Why is Department Match a feature?

[Easy]

Viva Answer: In hospitals, care providers are usually assigned to specific wards. Accessing records outside your department is a known indicator of insider snooping. But it can also happen legitimately in emergencies. The agent learns to combine dept_match=0 with other features to decide whether it's suspicious or urgent.

Q47. Why is Request Frequency divided into three categories?

[Hard]

Viva Answer: The three buckets – NORMAL (≤5/min), ELEVATED (6-20/min), BURST_HIGH (>20/min) – are based on clinical literature. A clinician treating a patient might make 5-6 record lookups per minute during an emergency. More than 20 per minute is characteristic of automated scripts and data scraping. These thresholds are domain-specific heuristics from Servida & Casey (2019).

Q48. Why is the state space finite and why does that matter?

[Medium]

Viva Answer: It's finite because all seven features are categorical – we discretized them. A finite, small state space means tabular Q-Learning converges exactly to the optimal Q-values, given enough exploration. With 1,152 states, the entire Q-table fits in memory as a 81 KB JSON file and each lookup is O(1).

Q49. What happens if you add one more role, say PHARMACIST?

[Hard]

Viva Answer: The Role dimension becomes 5. Total states = 5 × 2 × 4 × 2 × 3 × 3 × 2 = 1,440. The Q-table would become (1440, 4) = 5,760 values. You'd need to retrain from scratch and update the ROLES list in config.py, the ROLE_MAP, and the multipliers in state_to_index().

Quick Calculation: *5 × 2 × 4 × 2 × 3 × 3 × 2 = 1,440 states (288 more than current)*

Q50. Why discretize features instead of using continuous values?

[Medium]

Viva Answer: Continuous features would make tabular Q-Learning impossible because the state space would be infinite. We'd need function approximation like a neural network (DQN). For the current prototype, the discrete categories are clinically meaningful – 'external unknown' vs 'VPN offsite' are meaningfully different access contexts, not points on a continuous scale.

Q51. What is the memory requirement for the Q-table?

[Easy]

Viva Answer: 1,152 states × 4 actions × 8 bytes (float64) = 36,864 bytes = 36 KB in memory. The JSON file is ~81 KB because JSON adds string formatting overhead. Either way, it's trivially small.

Quick Calculation: *1152 × 4 × 8 bytes = 36,864 bytes ≈ 36 KB (in memory)*

Section 6: State Indexing – Deep Dive

The Linear State Index Formula:

```
Index(s) = Role × 288 + DeptMatch × 144 + Sensitivity × 36 + RequestTime × 18 + Location × 6 + Frequency × 2 + IsEmergency
```

Q52. Why do you convert the observation tuple to a single integer?

[Medium]

Viva Answer: NumPy arrays are indexed by integers. If I want to do Q[state], I need state to be a single integer index. A tuple like (1, 0, 2, 1, 2, 0, 1) can't directly index a 2D array. The bijection maps every unique tuple to a unique integer, giving O(1) array access.

Q53. Why not use a dictionary with tuple keys instead?

[Hard]

Viva Answer: Python dictionaries with tuple keys work but have higher memory overhead per entry and hash collision overhead. A NumPy array accessed by integer index is faster because it's a contiguous block of memory with O(1) offset arithmetic. At 1,152 states it makes no practical difference, but arrays are the standard approach for tabular RL.

Q54. Where does the multiplier 288 come from?

[Hard]

Viva Answer: 288 = DeptMatch × Sensitivity × RequestTime × Location × Frequency × Emergency = 2×4×2×3×3×2 = 288. It's the total size of the subspace for all features after Role. So each unit increase in Role skips past 288 states.

Quick Calculation: *2 × 4 × 2 × 3 × 3 × 2 = 288*

Q55. Where does the multiplier 144 come from?

[Hard]

Viva Answer: 144 = Sensitivity × RequestTime × Location × Frequency × Emergency = 4×2×3×3×2 = 144. It's the subspace size for all features after DeptMatch.

Quick Calculation: *4 × 2 × 3 × 3 × 2 = 144*

Q56. Where do 36, 18, 6, 2, 1 come from?

[Hard]

Viva Answer: 36 = 2×3×3×2 (subspace after Sensitivity). 18 = 3×3×2 (after RequestTime). 6 = 3×2 (after Location). 2 = 2 (after Frequency). 1 = the Emergency dimension itself. Each multiplier is the product of all remaining feature cardinalities to the right.

Quick Calculation: *36=2×3×3×2, 18=3×3×2, 6=3×2, 2=2, 1=1*

Q57. Give an example. State tuple (1,0,2,1,2,1,0) – what is its index?

[Hard]

Viva Answer: Role=1, DeptMatch=0, Sensitivity=2, RequestTime=1, Location=2, Frequency=1, IsEmergency=0. Index = 1×288 + 0×144 + 2×36 + 1×18 + 2×6 + 1×2 + 0 = 288 + 0 + 72 + 18 + 12

+ 2 + 0 = 392.

Quick Calculation: *1×288=288 + 0×144=0 + 2×36=72 + 1×18=18 + 2×6=12 + 1×2=2 + 0 = 392*

Q58. How do you decode index 392 back to its state tuple?

[Very Hard]

Viva Answer: Start with rem=392. IsEmergency = 392 % 2 = 0, rem = 392 // 2 = 196. Frequency = 196 % 3 = 1, rem = 196 // 3 = 65. Location = 65 % 3 = 2, rem = 65 // 3 = 21. RequestTime = 21 % 2 = 1, rem = 21 // 2 = 10. Sensitivity = 10 % 4 = 2, rem = 10 // 4 = 2. DeptMatch = 2 % 2 = 0, rem = 2 // 2 = 1. Role = 1 % 4 = 1. Result: (1,0,2,1,2,1,0). Matches.

Code Connection: *env.py → index_to_state(), line 97*

Q59. What is a bijection and why must the mapping be one-to-one?

[Very Hard]

Viva Answer: A bijection is a one-to-one and onto mapping – every state maps to exactly one index and every index maps to exactly one state. If two states mapped to the same index, the Q-learning updates for one state would corrupt the Q-values for the other. The bijection is guaranteed here because the weighted sum formula is the standard mixed-radix number representation.

Q60. Why is state indexing O(1)?

[Medium]

Viva Answer: The formula is just seven multiplications and six additions – a fixed number of arithmetic operations regardless of the number of states. No loops, no searching. Similarly, the array lookup Q[state_idx] is a single memory offset calculation in NumPy.

Section 7: Action Space

Q61. Why four actions instead of just ALLOW and DENY?

[Easy]

Viva Answer: Binary ALLOW/DENY is too coarse for healthcare. CHALLENGE lets us say 'I'm not sure – prove you're really you' without outright blocking. RESTRICT lets us say 'you can access, but only the minimum necessary view' – important for cross-department access or researchers. Intermediate actions reduce both false denials and unnecessary full access.

Q62. How is RESTRICT different from DENY?

[Easy]

Viva Answer: DENY completely blocks the request – no data is returned. RESTRICT returns a masked or de-identified view. For example, vital signs but not psychiatric notes. This is the 'minimum necessary' principle from HIPAA. It's particularly useful for insider misuse – you give them something but hide the sensitive parts.

Q63. How is CHALLENGE different from ALLOW?

[Medium]

Viva Answer: ALLOW immediately grants full access. CHALLENGE says 'provide step-up verification first – an OTP, biometric, or supervisor override'. In the prototype, this is returned as a decision label; no actual MFA is integrated. In a real system, CHALLENGE would redirect the user to an MFA flow before access is granted.

Q64. Is CHALLENGE actually connected to MFA in this prototype?

[Medium]

Viva Answer: No. The code returns 'CHALLENGE' as an action string with a description saying 'Step-up 2FA/Biometric verification'. But there's no actual MFA integration in the codebase. It's a prototype decision label. In a real deployment, CHALLENGE would trigger an MFA redirect.

Note: *Not directly verified as integrated with any MFA service from the supplied implementation.*

Q65. Is RESTRICT enforcing actual field-level masking?

[Medium]

Viva Answer: No. The API returns 'RESTRICT' as an action string. The frontend might show a masked modal, but the actual masking of specific fields in medical records is not implemented in the backend. This is a research prototype demonstrating the decision logic, not a fully integrated clinical system.

Note: *Not directly verified as enforcing field-level masking from the supplied implementation.*

Q66. For STOLEN_CREDENTIAL, why is CHALLENGE rewarded with +14 but DENY gets +18?

[Hard]

Viva Answer: Both block the attacker, but DENY is more decisive. CHALLENGE is still valuable because an automated attack script can't complete MFA verification, so it effectively blocks the attack too. The slightly lower reward for CHALLENGE reflects that there's still a small window

where a sophisticated attacker might attempt to socially engineer the MFA step.

Q67. Why does ALLOW during MASS_ACCESS get the maximum penalty of -35?

[Hard]

Viva Answer: Mass access represents automated bulk exfiltration. A single ALLOW response could expose thousands of records if the scraper is running at burst frequency. That's catastrophic – hence the same penalty as denying a life-saving emergency (-35). The agent must learn that mass-scraping requests are as dangerous as emergency denials, just in the opposite direction.

Section 8: Reward Function

Complete Reward Matrix (Verified from env.py → calculate_reward())

Scenario	ALLOW	DENY	CHALLENGE	RESTRICT
EMERGENCY_ACCESS	+20.0 ✓	-35.0 ✗ (worst)	-10.0 ✗	-5.0 ✗
NORMAL_ACCESS	+12.0 ✓	-18.0 ✗	+1.0 ±	+3.0 or -3.0*
STOLEN_CREDENTIAL	-30.0 ✗ (breach)	+18.0 ✓	+14.0 ✓	-8.0 ✗
INSIDER_MISUSE	-22.0 ✗ (breach)	+15.0 ✓	+6.0 ✓	+8.0 ✓
MASS_ACCESS	-35.0 ✗ (worst)	+20.0 ✓	+8.0 ✓	-10.0 ✗

** NORMAL+RESTRICT: +3.0 if dept_match=0; -3.0 if dept_match=1 AND role in {DOCTOR, NURSE} (verified from env.py line 321-327)*

Q68. Why does EMERGENCY_ACCESS DENY get -35 – the maximum negative reward?

[Medium]

Viva Answer: Because denying a doctor access to vital records during a cardiac arrest or trauma can directly cause patient death. That's the worst possible outcome in a hospital system. The agent must learn that denying emergency access is catastrophic – hence the highest possible penalty.

Code Connection: *env.py → calculate_reward(), line 305: reward = -35.0 when scenario==EMERGENCY and action==DENY*

Q69. Why does EMERGENCY_ACCESS CHALLENGE get -10?

[Medium]

Viva Answer: Because even asking for MFA during a cardiac arrest wastes precious seconds. Every delay in an emergency can worsen patient outcomes. CHALLENGE is not as bad as DENY (-35) because the user can theoretically still complete it and get access, but it's still wrong.

Q70. Why does EMERGENCY_ACCESS RESTRICT get -5?

[Medium]

Viva Answer: Masked vital signs during a trauma event could hide critical information the doctor needs. RESTRICT is the least-bad wrong answer for an emergency – the clinician at least gets some data, just potentially incomplete.

Q71. Why does NORMAL_ACCESS DENY get -18 but EMERGENCY DENY gets -35?

[Medium]

Viva Answer: The severity of the clinical harm is lower in routine access. If a routine access is denied, the clinician is inconvenienced and can use other channels. If an emergency access is denied, there may be no time for alternatives. The asymmetry reflects the urgency.

Q72. Why does NORMAL_ACCESS RESTRICT get +3 for dept_match=0 but -3 for a Doctor with dept_match=1?

[Hard]

Viva Answer: If the requester is from a different department, restricting their view is a reasonable precaution even for a legitimate request – they get minimum-necessary access. But if the requester is a Doctor in their own department and you restrict them unnecessarily, that's clinical friction with no benefit. The reward function captures this nuance.

Code Connection: *env.py → calculate_reward(), lines 321-327*

Q73. Why is STOLEN_CREDENTIAL ALLOW penalized -30 but not -35?

[Hard]

Viva Answer: A stolen credential accessing one record is a serious breach (-30) but slightly less catastrophic than a mass scraping attack (+35 penalty for ALLOW on MASS_ACCESS), which could expose bulk patient data. The scale reflects volume of potential harm.

Q74. Why does INSIDER_MISUSE RESTRICT get +8 – a positive reward?

[Hard]

Viva Answer: Insider snooping on VIP records is an attempt to access data outside clinical need. RESTRICT gives the insider a de-identified view, protecting the VIP's sensitive information while not triggering a complete lockout that might alert a sophisticated insider. It's a containment strategy that's clinically reasonable and legally defensible under HIPAA minimum-necessary principles.

Q75. Sir says: your reward function is manually designed, so isn't the agent just memorizing your rules?

[Very Hard]

Viva Answer: That's a fair point. The reward function does encode domain knowledge about what is good or bad. But the key difference from a rule engine is that the agent discovers the right action for each specific state combination automatically through experience, not from hard-coded if-else rules. For example, the agent learns that RESTRICT is good for insider misuse but bad for mass access – that context-sensitive nuance is not a rule I explicitly programmed. The reward is designed; the policy is learned.

Trap: *The examiner wants to check if you understand that RL agents can still add value even with designed rewards.*

Q76. Does the agent see the scenario_type?

[Medium]

Viva Answer: Absolutely not. The agent only sees the Observation namedtuple – role, dept_match, sensitivity, time, location, frequency, emergency. The scenario and is_legitimate are in hidden_meta which is never passed to choose_action() or update(). The scenario is only used by calculate_reward() inside the environment.

Code Connection: *env.py → generate_request() returns (obs, hidden_meta); only obs goes to the agent*

Q77. Is using hidden scenario labels during reward calculation 'cheating'?

[Very Hard]

Viva Answer: It's standard RL practice. The environment can have ground truth that the agent never sees. The agent must learn to infer the right action from observable features alone. In real deployment, there's no hidden_meta at all – the agent just sees the observable context and applies its learned policy. The hidden labels are only needed for training signal generation.

Q78. Is the reward sparse or dense?

[Medium]

Viva Answer: Dense. Every single action in every single episode receives a non-zero reward. The agent gets immediate feedback after every decision. Sparse rewards would be something like '+100 only if the system remains unbreached for a full year'. Dense rewards make learning faster.

Q79. Is the reward deterministic?

[Medium]

Viva Answer: Given the same scenario and the same action, yes. The reward function is a fixed lookup table – EMERGENCY + ALLOW always gives +20. However, different episodes can produce different rewards because the scenario itself is randomly sampled by the environment.

Section 9: MDP Theory

Q80. Why is this problem modeled as an MDP?

[Medium]

Viva Answer: An MDP requires: states, actions, a transition model, a reward function, and a discount factor. My project has all of these. States are the 7-tuple observations. Actions are the four decisions. Rewards come from the piecewise reward matrix. Transitions are the environment generating the next request. Discount factor gamma=0.90. So it fits the MDP definition.

Q81. Is your MDP truly Markov?

[Very Hard]

Viva Answer: Mostly. The Markov property says the future depends only on the current state, not history. My state vector captures the current request's context. However, frequency – one of the seven features – implicitly encodes some history. Strictly speaking, if we needed to track the entire sequence of past requests to determine frequency, the state would not be fully Markov. In our discrete encoding, the frequency is a pre-computed category, so within that abstraction it is Markov.

Q82. What is P(s'|s,a) in your project?

[Hard]

Viva Answer: The transition probability is the probability of moving to next state s' given current state s and action a. In my project, because each episode is one-step and done=True, the next state is simply the next request sampled by generate_request(). The transition is independent of the current state and action – it's driven purely by the scenario distribution weights [0.50, 0.15, 0.12, 0.12, 0.11].

Code Connection: *env.py → generate_request(), line 134: random.choices with weights*

Q83. Is your environment deterministic or stochastic?

[Medium]

Viva Answer: Stochastic. The scenario is randomly sampled each episode. Within a scenario, feature values are also randomly chosen with probabilities (e.g., in NORMAL_ACCESS, dept_match is 1 with probability 0.92). So different episodes can produce different observations even under the same seed after it diverges.

Q84. What is the discount factor gamma and why 0.90?

[Medium]

Viva Answer: Gamma determines how much future rewards are valued. At gamma=0.90, a reward one step in the future is worth 0.90 of its face value now. Since each episode is effectively one-step with done=True, gamma doesn't have a strong effect in our case. But we keep it at 0.90 for theoretical correctness and because if episodes were ever extended to multi-step, the value would make sense.

Q85. What is the Bellman optimality equation?

[Medium]

Viva Answer: Q*(s,a) = E[r + gamma × max_a' Q*(s', a') | s, a]. It says the optimal Q-value for state s and action a equals the immediate reward plus the discounted best future Q-value. It's a recursive definition that the Q-learning update converges to.

Q86. What is the difference between a value function V(s) and a Q-function Q(s,a)?

[Medium]

Viva Answer: V(s) is the expected total reward from state s, following the optimal policy. Q(s,a) is the expected total reward from state s, taking action a first, then following the optimal policy. Q(s,a) is more useful for control because it tells you which action to take. V(s) doesn't tell you that directly unless you also know the transition model.

Section 10: Q-Learning Theory

Q-Learning Update Rule:

```
Q(s,a) ← Q(s,a) + α × [r + γ × max_a' Q(s',a') – Q(s,a)]
```

For one-step terminal episodes (done=True), this simplifies to:

```
Q(s,a) ← Q(s,a) + α × [r – Q(s,a)]
```

Q87. Is Q-Learning model-free or model-based?

[Easy]

Viva Answer: Model-free. It doesn't learn or use a transition model P(s'|s,a) or a reward model. It directly updates Q-values from experience – actual rewards and states observed during interaction. It never tries to predict what will happen next; it just records what did happen.

Q88. Is Q-Learning on-policy or off-policy?

[Medium]

Viva Answer: Off-policy. The behavior policy during training is epsilon-greedy – it takes random actions some of the time. But the learning target uses max_a' Q(s',a') – the greedy optimal action, not the action that was actually taken. So it learns the optimal (greedy) policy even while behaving non-optimally during exploration.

Q89. Why is off-policy a useful property?

[Hard]

Viva Answer: It means we can separate the data collection policy from the target policy. We can use any exploration strategy – epsilon-greedy, softmax, random – to generate training data, and still converge to the optimal Q-values. This makes Q-Learning more flexible than on-policy methods like SARSA.

Q90. What is the TD error, term by term?

[Medium]

Viva Answer: TD stands for Temporal Difference. TD error = [r + γ × max Q(s',a')] – Q(s,a). The first bracket is the 'target' – what Q(s,a) should approximately equal. The second part Q(s,a) is the current estimate. The difference tells us how wrong we were. We then move Q(s,a) toward the target by a fraction alpha of the error.

Code Connection: *agent.py → update(), lines 83-93: td_error = target - current_q*

Q91. What is bootstrapping in Q-Learning?

[Hard]

Viva Answer: Bootstrapping means using your own current estimate Q(s',a') to compute the update target, rather than waiting for the actual return. In Monte Carlo, you'd wait until the episode ends and use the real cumulative reward. In TD/Q-Learning you update immediately using an estimate. It's faster but introduces some bias.

Q92. Why not SARSA instead of Q-Learning?

[Medium]

Viva Answer: SARSA is on-policy – it updates using the action that was actually taken, including random actions. Q-Learning always updates using the max action. For my project, I want the agent to learn the optimal policy, not an epsilon-greedy policy. Q-Learning converges to the optimal policy regardless of the behavior strategy, making it better suited here.

Q93. What is alpha and why is it 0.10?

[Easy]

Viva Answer: Alpha is the learning rate – it controls how much each new experience updates the Q-value. At alpha=0.1, the agent moves 10% toward the new target estimate each update. 0.1 is a standard conservative value. Too high (e.g. 1.0) and the agent forgets previous learning on every update; too low and it converges very slowly.

Code Connection: *agent.py → __init__(), line 36: learning_rate=0.1*

Q94. What happens if alpha = 1?

[Hard]

Viva Answer: The update becomes Q(s,a) = r + γ × max Q(s',a'). The entire old value is thrown away and replaced by the new target. The Q-table becomes very noisy because each stochastic reward immediately overwrites the accumulated knowledge. The agent will fail to converge in a stochastic environment.

Q95. What happens if alpha = 0?

[Easy]

Viva Answer: The Q-table never updates – Q(s,a) += 0. The agent can never learn anything. It would behave randomly during exploration and then always select whatever action was initialized (all zeros → random tie-breaking) during inference.

Q96. What happens if gamma = 0?

[Medium]

Viva Answer: The agent ignores all future rewards and only cares about the immediate reward. The update becomes Q(s,a) ← Q(s,a) + alpha × [r - Q(s,a)]. For one-step episodes this is fine, but for multi-step problems the agent becomes myopic and fails to plan ahead.

Q97. What happens if gamma = 1?

[Hard]

Viva Answer: Future rewards are valued equally to immediate rewards – no discounting. This can cause convergence problems in continuing (non-episodic) tasks because the sum of rewards may become infinite. For episodic tasks it's sometimes okay but can slow convergence.

Q98. Under what conditions does tabular Q-learning converge to the optimal Q*?

[Very Hard]

Viva Answer: Three conditions: (1) All state-action pairs must be visited infinitely often – guaranteed by epsilon>0. (2) The learning rate alpha must decay appropriately such that sum(alpha_t) diverges but sum(alpha_t^2) converges. In practice, constant alpha with enough training works empirically. (3) The rewards must be bounded. All three are satisfied in our discrete

environment.

Q99. Why not use DQN (Deep Q-Network)?

[Medium]

Viva Answer: DQN is needed when the state space is too large for a table – like Atari pixel images. My state space is 1,152 states, tiny by any standard. A neural network would be overkill, would require far more data, is harder to interpret, and has no convergence guarantee for our small discrete problem. Tabular Q-Learning is optimal here.

Section 11: Epsilon-Greedy Exploration

Q100. What is epsilon-greedy and why do we need it?

[Easy]

Viva Answer: Epsilon-greedy is a simple exploration strategy. With probability epsilon, pick a random action; otherwise, pick the greedy best action. We need it because if the agent always exploits what it knows, it might never discover that some unexplored action could be better.

Code Connection: *agent.py → choose_action(), line 62: if np.random.rand() < self.epsilon*

Q101. Why start epsilon at 1.0?

[Easy]

Viva Answer: At the beginning, the Q-table is all zeros – the agent knows nothing. Starting at epsilon=1.0 ensures that for the first episodes the agent explores completely randomly, visiting different states and actions to build up initial knowledge before starting to exploit.

Q102. Why is the minimum epsilon 0.01 and not 0?

[Easy]

Viva Answer: Keeping a 1% chance of random actions prevents the agent from prematurely locking in on a suboptimal policy. It maintains a tiny floor of ongoing exploration in case the environment changes slightly or rare states haven't been visited enough.

Code Connection: *agent.py → decay_epsilon(), line 99: max(self.epsilon_min, ...)*

Q103. Why is the decay 0.9998?

[Hard]

Viva Answer: With decay=0.9998, epsilon falls as: ε_k = max(0.01, 1.0 × 0.9998^k). After 35,000 episodes: 0.9998^35000 ≈ 0.0009, which is below 0.01, so epsilon clamps at 0.01. This ensures the agent transitions from mostly-exploration to mostly-exploitation over the full training run, roughly reaching the minimum around episode 23,000.

Quick Calculation: *0.9998^35000 ≈ e^(35000×ln(0.9998)) ≈ e^(-7.0) ≈ 0.0009 < 0.01*

Q104. ■ IMPORTANT DISCREPANCY: What is the actual epsilon_decay in the code?

[Very Hard]

Viva Answer: The agent.py constructor default is epsilon_decay=0.9997. But train.py's train_agent() function explicitly passes epsilon_decay=0.9998 to the agent. The dossier and all reported training results use 0.9998. So the effective value during the documented training run is 0.9998.

Note: *If a professor looks at agent.py line 40 and asks 'why 0.9997?', clarify that train.py overrides the default with 0.9998 at line 30.*

Q105. Calculate epsilon after episode 5,000 using decay=0.9998.

[Hard]

Viva Answer: ε = max(0.01, 1.0 × 0.9998^5000). 0.9998^5000 = e^(5000 × ln(0.9998)) ≈ e^(5000 × (-0.0002)) = e^(-1.0) ≈ 0.3679. The dossier reports 0.3678 at episode 5,000 – consistent with this

calculation.

Quick Calculation: *0.9998^5000 ≈ e^(-1.0) ≈ 0.3679*

Q106. Is exploration used during inference?

[Easy]

Viva Answer: No. During inference, choose_action() is called with training=False. The epsilon-greedy branch is skipped entirely. The agent always picks the greedy argmax action.

Code Connection: *agent.py → choose_action(), line 62: if training and np.random.rand() < self.epsilon*

Q107. What happens if epsilon never decays?

[Medium]

Viva Answer: The agent keeps exploring randomly 100% of the time and never exploits its learned knowledge. The Q-table would still be updated, but the agent's training-time behavior would always be random. This is fine for Q-learning convergence in theory (with enough steps) but would make the agent very slow to learn.

Section 12: Training

Q108. Why 35,000 episodes?

[Medium]

Viva Answer: With 1,152 states and 4 actions, there are 4,608 state-action pairs. Each episode visits one pair. 35,000 episodes gives approximately 30 visits per state-action pair on average – enough for Q-values to stabilize. The actual convergence evidence is the declining TD error reaching 0.4553 by episode 35,000.

Q109. Is one access request one episode?

[Easy]

Viva Answer: Yes. The environment sets done=True after every action in env.step(). So each episode is exactly one request, one action, one reward. There's no multi-step trajectory.

Code Connection: *env.py → step(), line 252: done = True*

Q110. Why does the TD target simplify to just the reward in your case?

[Hard]

Viva Answer: Because done=True every episode. In the update() function, when done is True, the target is just the reward: target = reward. The max Q(s',a') term is discarded because there's no meaningful next state to bootstrap from. The Q-update becomes: Q(s,a) += alpha × (reward - Q(s,a)).

Code Connection: *agent.py → update(), lines 85-89: if done: target = reward*

Q111. Why does average reward increase during training?

[Medium]

Viva Answer: Initially epsilon is near 1.0, so the agent mostly picks random actions including bad ones. As epsilon decays and Q-values become meaningful, the agent increasingly picks better actions. By episode 35,000, epsilon is 0.01, so 99% of actions are greedy-optimal and the average reward stabilizes at +14.07.

Q112. Why use seed 42 for training and seed 100 for evaluation?

[Medium]

Viva Answer: Different seeds ensure the training and test data are statistically independent. If the same seed were used, the model might memorize the exact training scenarios. Seed 42 is a conventional choice (the 'answer to everything'). Seed 100 for evaluation generates a completely different sequence of 5,000 requests that the agent has never seen during training.

Q113. Why is tabular training completed in ~1.27 seconds?

[Medium]

Viva Answer: Because each episode is just: generate request (random sampling), compute state index (7 multiplications), array lookup + argmax (NumPy), one reward lookup, and one Q-table update. No gradient computation, no neural network forward/backward pass. NumPy reports ~27,521 episodes per second. DQN on the same hardware would take minutes or hours.

Section 13: Training Results & Convergence

Episode	Avg Reward	Epsilon	TD Error	Interpretation
1,000	-1.39	0.8187	11.2597	Mostly random; frequent bad actions; high uncertainty
5,000	+7.10	0.3678	5.1999	Transition: learning kicks in, policy improves
15,000	+12.82	0.0498	1.8045	Near-greedy; policy substantially converged
25,000	+13.86	0.0100	0.7370	Epsilon at minimum; fine-tuning Q-values
35,000	+14.07	0.0100	0.4553	Final trained state; highly stable policy

Q114. Why is average reward negative at episode 1,000?

[Medium]

Viva Answer: At episode 1,000, epsilon is still 0.8187. So about 82% of actions are random. Random actions often produce heavily penalized decisions – like DENY during emergency (-35) or ALLOW during stolen credentials (-30). These dominate the average and pull it to -1.39.

Q115. Why does epsilon reach exactly 0.01 and stop at episode 25,000?

[Hard]

Viva Answer: The decay is multiplicative: epsilon = max(0.01, epsilon × 0.9998). Once epsilon hits 0.01, the max() clamps it there permanently. This happens around episode 23,000 (ln(0.01)/ln(0.9998) ≈ 23,026). After that, epsilon stays at 0.01 for the remaining ~12,000 episodes.

Quick Calculation: *ln(0.01) / ln(0.9998) ≈ -4.605 / (-0.0002) ≈ 23,026 episodes to reach minimum*

Q116. Does decreasing TD error prove convergence?

[Very Hard]

Viva Answer: It's strong evidence but not proof. Decreasing TD error means the Q-values are stabilizing – the gap between estimates and targets is shrinking. But to formally prove convergence, you'd need to show the Q-values are no longer changing significantly and match the theoretical optimum. In practice, the combination of decreasing TD error and plateauing reward is the standard empirical convergence signal.

Q117. Why does TD error not reach exactly 0.0?

[Hard]

Viva Answer: Because epsilon=0.01 means 1% of actions are still random. Random actions produce non-optimal rewards, creating persistent TD error. Also the environment is stochastic – the same state can generate different scenarios, creating irreducible variance in rewards.

Section 14: Attack Simulation

Q118. What does NORMAL_ACCESS represent and what features characterize it?

[Easy]

Viva Answer: It represents routine legitimate clinical work. The scenario weights favor Doctors and Nurses, 92% probability of department match, no emergency flag, mostly normal frequency, mostly internal hospital location, mostly during normal hours. It's the baseline of legitimate hospital activity making up 50% of traffic.

Code Connection: *env.py → generate_request(), lines 140-149*

Q119. What makes EMERGENCY_ACCESS distinctive?

[Medium]

Viva Answer: Emergency requests have is_emergency=1, are more likely to be off-hours (50/50 split), more likely to have department mismatch (55% probability), and frequently access HIGH or CRITICAL_VIP records. The key challenge is that a legitimate emergency looks suspicious in some dimensions – cross-department, off-hours, high-sensitivity – which a naive system would deny.

Code Connection: *env.py → generate_request(), lines 151-163*

Q120. What makes STOLEN_CREDENTIAL attacks detectable?

[Medium]

Viva Answer: Three features: external location (85% EXTERNAL_UNKNOWN), off-hours (80% probability), and high/VIP sensitivity (they target valuable records). No emergency flag (90% no-emergency). The combination of external + off-hours + high-sensitivity is a strong attack signal that the agent learns to trigger DENY or CHALLENGE on.

Code Connection: *env.py → generate_request(), lines 165-176*

Q121. How is INSIDER_MISUSE different from STOLEN_CREDENTIAL?

[Hard]

Viva Answer: Insider misuse comes from inside the hospital (90% INTERNAL_HOSPITAL) during working hours (60%). The attacker has legitimate credentials from their actual role. The distinguishing feature is dept_match=0 (always – hardcoded) and CRITICAL_VIP sensitivity (75%). No burst frequency, no emergency. It's harder to detect because the location and timing look normal.

Code Connection: *env.py → generate_request(), lines 178-187: dept_match = 0 always*

Q122. What makes MASS_ACCESS unique in the feature space?

[Medium]

Viva Answer: frequency=BURST_HIGH is hardcoded – always >20 queries/min. This is the dead giveaway. Combined with external location (70%) and dept_match=0 (always), the agent learns to DENY immediately. No legitimate use case generates burst frequency from external unknown sources.

Code Connection: *env.py → generate_request(), line 196: frequency = FREQUENCY_MAP['BURST_HIGH']*

Q123. Are these real attacks? Did you hack anything?

[Easy]

Viva Answer: No. All data is entirely synthetic, generated by a Python simulation function (generate_request() in env.py). The attack patterns are modeled based on academic literature – Mavridis et al. on insider snooping, Servida & Casey on burst frequency scraping. No real hospital systems were accessed or tested.

Q124. Why use synthetic data instead of real EHR logs?

[Medium]

Viva Answer: Real EHR data contains Protected Health Information (PHI) covered by HIPAA/GDPR. Accessing real patient records without proper authorization and data use agreements would be illegal and unethical. Synthetic data lets us simulate realistic patterns without privacy concerns.

Section 15: Database

Table	Primary Key	Foreign Keys	Key Columns	Row Count
users	user_id (int)	department_id → departments	username, full_name, role, usual_location, is_active	103
patients	patient_id (int)	department_id → departments	patient_code, full_name, age, gender	1,000
medical_records	record_id (int)	patient_id → patients	record_type, sensitivity, record_count	3,548
departments	department_id (int)	None	department_name (5 depts)	5
access_requests	request_id (int)	user_id, patient_id, record_id	location, emergency_status, department_match, scenario_type	10,052+
rl_decisions	decision_id (int)	request_id → access_requests	algorithm, action, reward, decision_time_ms	18+
audit_logs	audit_id (int)	request_id, user_id	action, event_description, logged_at	48+

Q125. Why separate access_requests and rl_decisions into two tables?

[Medium]

Viva Answer: Separation of concerns. access_requests logs what was requested – the telemetry from the user. rl_decisions logs what the agent decided – algorithm, action, reward, latency. This lets you query them independently: audit which requests were made, or analyze only the RL decisions without the full request context.

Q126. Why keep audit_logs as a separate table?

[Medium]

Viva Answer: Audit logs serve a different purpose – compliance and forensics. They need to be immutable in a real system. Separating them allows different access control policies (audit_logs might be append-only for all users but readable only by security administrators).

Q127. Does the RL agent directly query the database?

[Medium]

Viva Answer: No. The agent only uses its in-memory Q-table for decisions. The database is only written to after the decision is made – it stores the audit trail. The agent's decision logic is completely independent of the database.

Q128. What happens if Supabase is unavailable?

[Medium]

Viva Answer: The log_access_and_decision() function catches all exceptions and returns False. The API still returns the RL decision correctly – the Supabase failure only means the decision is not persisted to the database. The logged_to_supabase field in the response will be False.

Code Connection: *main.py → evaluate_access_request(), lines 235-254: try/except around log_access_and_decision*

Q129. Why does db.py inspect the schema dynamically instead of hardcoding columns?

[Hard]

Viva Answer: The requirement explicitly states: 'Do not assume database columns'. The Supabase schema may have been modified by the DBA independently. Dynamic inspection

ensures the code doesn't break when a column is renamed or a new one is added. adapt_payload_to_schema() then filters the payload to only include columns that actually exist.

Code Connection: *db.py → inspect_schema(), line 47*

Section 16: FastAPI / Backend

Endpoint	Method	Purpose
/	GET	Serves index.html SPA or returns API info
/health	GET	System health, model loaded status
/db/test	GET	Supabase connection test and table audit
/schema	GET	Dynamic column schema of all 7 tables
/predict	POST	Main RL decision engine (ALLOW/DENY/CHALLENGE/RESTRICT)
/simulate	GET	Generate and evaluate a synthetic request with ground truth
/metrics	GET	Reads saved evaluation_results.json
/api/users	GET	Fetch users from Supabase (or mock fallback)
/api/patients	GET	Search patients with optional filters
/api/patients/{id}/records	GET	Fetch medical records for a patient
/api/history	GET	Access requests joined with RL decisions
/api/stats	GET	Dashboard KPIs, model metrics, DB counts

Q130. Why is /predict a POST and not a GET?

[Medium]

Viva Answer: HTTP GET requests should be idempotent and have no side effects. /predict mutates state – it writes to the Supabase database. Also, the request body contains a JSON payload which is not appropriate for URL query parameters. POST is the correct HTTP method for submitting data and triggering side effects.

Q131. What data does /predict receive and return?

[Medium]

Viva Answer: It receives an AccessRequestSchema JSON body: user_id, patient_id, record_id, role, department_match, record_sensitivity, request_time, location, request_frequency, emergency_status. It returns a DecisionResponseSchema: status, action, action_description, q_values, explanation, decision_time_ms, request_id, timestamp, logged_to_supabase.

Code Connection: *main.py → AccessRequestSchema line 86, DecisionResponseSchema line 101*

Q132. How does FastAPI validate the incoming request?

[Medium]

Viva Answer: Pydantic validates the request body automatically. Each field in AccessRequestSchema has a type annotation. If a field fails validation (e.g., wrong type), FastAPI returns HTTP 422 Unprocessable Entity with a detailed error message before the function even executes.

Q133. What happens if an invalid role like 'ALIEN' is provided?

[Hard]

Viva Answer: FastAPI doesn't reject it at the Pydantic level because role is just a str type with no enum constraint. parse_context_to_observation() in env.py calls ROLE_MAP.get(raw_role, ROLE_MAP['DOCTOR']). Since 'ALIEN' is not in ROLE_MAP, it falls back to DOCTOR (0). The API will process the request successfully with DOCTOR as the role.

Code Connection: *env.py → parse_context_to_observation(), line 413: ROLE_MAP.get(raw_role, ROLE_MAP['DOCTOR'])*

Q134. How is inference latency measured?

[Medium]

Viva Answer: Two time.perf_counter() calls bracket just the discretization + argmax computation: t0 before parse_context_to_observation() and t1 after choose_action(). The difference (t1-t0) × 1000 gives milliseconds. This measures pure model inference time, NOT total API round-trip time.

Code Connection: *main.py → evaluate_access_request(), lines 213-225*

Q135. Can someone bypass the frontend and call /predict directly?

[Hard]

Viva Answer: Yes, absolutely. The API has no authentication. Anyone who can reach port 8000 can POST to /predict with any JSON body. CORS is set to allow_origins=['*']. This is acceptable for a research prototype but would need JWT authentication and API keys in a production hospital deployment.

Code Connection: *main.py → CORSMiddleware, line 43: allow_origins=['*']*

Q136. Is the Q-table modified during inference?

[Easy]

Viva Answer: No. choose_action() with training=False is pure read-only – it reads Q[state_idx] and returns the argmax. The update() function is never called during API requests. The Q-table is effectively immutable once loaded.

Code Connection: *agent.py → choose_action(), lines 57-69: no q_table writes*

Q137. /simulate vs /predict – what is the key difference?

[Medium]

Viva Answer: Both run the agent. But /simulate generates a synthetic request from the environment itself, meaning it has access to hidden_meta (scenario, is_legitimate) and calculates the actual reward. It returns the ground truth alongside the decision. /predict accepts a real user-submitted request with no ground truth – it just makes the decision and logs it.

Code Connection: *main.py → simulate_scenario(), line 269 vs evaluate_access_request(), line 207*

Section 17: Frontend

Q138. What does the frontend do?

[Easy]

Viva Answer: The frontend is a Single Page Application with 6 dashboards: a hospital staff view showing KPIs, a patient records browser, the RL decision engine form, a security admin dashboard with Chart.js visualizations, an HTML5 Canvas cyber attack simulator showing network packet animations, and an audit history table. It communicates with the FastAPI backend via JavaScript fetch() calls.

Q139. How does the frontend call /predict?

[Easy]

Viva Answer: In static/js/access.js, a fetch() POST request is made to '/predict' with the form data serialized as JSON. The response JSON is parsed and used to update the UI – the action badge, Q-value bar chart, and explanation text.

Code Connection: *static/js/access.js (JS file handles the fetch POST)*

Q140. Is the Canvas network visualization part of the RL algorithm?

[Easy]

Viva Answer: No. It's a pure UI demonstration showing packet routing animations between hospital network nodes. It's a visual representation of the simulation concept but has no connection to the Q-table or the actual RL decision logic.

Code Connection: *static/js/simulator.js*

Trap: *The professor might want you to confuse the animation with the actual RL computation.*

Q141. Why HTML/CSS/JavaScript instead of React/Vue?

[Easy]

Viva Answer: Vanilla JS/CSS requires no build system, no npm, no webpack. For a prototype dashboard that needs to be simple, deployable, and demonstrable in an academic setting, vanilla tech is faster to build and has zero dependency risk. A framework would add complexity without benefit for this scale.

Q142. Does the frontend provide any security?

[Easy]

Viva Answer: No. Browser-side JavaScript can be modified by anyone. The CORS configuration allows all origins. All security decisions happen in the backend Q-table lookup. The frontend is purely a UI wrapper.

Trap: *Professor might ask this to see if you think the frontend is a security boundary – it is not.*

Section 18: Database + RL Integration

Q143. What is the dependency between the RL model and the database?

[Medium]

Viva Answer: The RL model has zero dependency on the database at inference time. The Q-table lookup happens entirely in memory. The database is written to after the decision. You could remove Supabase entirely and the RL decision would still work perfectly – you'd just lose the audit trail.

Q144. Could you run the RL model without the database?

[Easy]

Viva Answer: Yes. The API would still return correct ALLOW/DENY/CHALLENGE/RESTRICT decisions. The logged_to_supabase field would be False. The model and the database are architecturally decoupled.

Q145. Does the reward stored in rl_decisions come from the environment reward function?

[Very Hard]

Viva Answer: No, and this is subtle. In db.py log_access_and_decision(), the reward stored is: float(q_values.get(action, 0.0)) – it's the Q-value of the selected action, not the actual reward from calculate_reward(). The actual reward would require hidden_meta which is not available during real-time inference.

Code Connection: *db.py → log_access_and_decision(), line 388: reward_val = float(q_values.get(action, 0.0))*

Trap: *The stored 'reward' in the DB is actually the Q-value, not the environment reward.*

Section 19: Evaluation

Q146. Why evaluate on 5,000 requests and not 1,000?

[Medium]

Viva Answer: More samples reduce variance in the metric estimates. With 5,000 requests and scenario weights [0.50, 0.15, 0.12, 0.12, 0.11], we get roughly 750 emergency requests and 600 stolen credentials – enough to get statistically stable attack blocking and emergency denial rates.

Q147. What does 99.37% attack blocking rate actually mean?

[Hard]

Viva Answer: It means: of all simulated attack requests (STOLEN_CREDENTIAL + INSIDER_MISUSE + MASS_ACCESS), 99.37% were blocked by either DENY or CHALLENGE – or RESTRICT for INSIDER_MISUSE. It does NOT mean 99.37% of all 5,000 requests were blocked. Only attack-type requests are in the denominator.

Code Connection: *evaluate.py → line 128: attack_blocking_rate = attacks_blocked / total_attacks × 100*

Trap: *The trap is thinking the denominator is all requests. It's only attack-type requests.*

Q148. What does 99.79% legitimate access rate mean?

[Hard]

Viva Answer: Of all legitimate requests (NORMAL_ACCESS + EMERGENCY_ACCESS), 99.79% received ALLOW, CHALLENGE, or RESTRICT – i.e., some form of access was granted. Only 0.21% were denied outright. Note that for legitimate requests, CHALLENGE and RESTRICT both count as 'granted' in the numerator.

Code Connection: *evaluate.py → lines 117-118: legitimate_granted counted if action in (ALLOW, CHALLENGE, RESTRICT)*

Q149. Why is emergency denial rate especially important?

[Medium]

Viva Answer: Emergency denial is life-threatening. Even 1% of emergency requests being denied could mean a doctor cannot access critical patient vitals during a Code Blue. The 0.13% rate means in 5,000 simulated requests, out of ~772 emergencies, only 1 was denied. That 1 case represents a 1-in-772 failure rate for the most critical scenario.

Q150. What is p95 latency and why report it?

[Medium]

Viva Answer: p95 means the 95th percentile decision latency – 95% of decisions are faster than 0.0222 ms. Reporting p95 alongside the mean (0.0184 ms) shows that latency is consistent with few outliers. If we only reported mean, a handful of slow outliers could mask performance issues for rare states.

Code Connection: *evaluate.py → line 133: np.percentile(decision_times_ms, 95)*

Q151. Is the decision latency the same as API round-trip latency?

[Hard]

Viva Answer: No. The 0.0184 ms is the pure RL inference time – just the state encoding and argmax. The actual API round-trip includes network overhead, JSON parsing, Pydantic validation, the Supabase write, and JSON serialization. That would be tens to hundreds of milliseconds.

Trap: *Professor will try to catch you claiming 0.018ms is the total API latency. It is not.*

Q152. Is there data leakage in your evaluation?

[Medium]

Viva Answer: No. The training environment uses seed=42 and the evaluation environment uses seed=100. These generate independent sequences of requests. The Q-table was built entirely from the training data and evaluated on fresh, previously unseen scenarios.

Q153. What does cumulative reward +71,363 tell us?

[Hard]

Viva Answer: It's the sum of rewards across all 5,000 test requests. A strongly positive cumulative reward means the agent is consistently making decisions that the reward function rates as good. It's a composite signal but depends entirely on the reward design. It doesn't independently validate security – it shows the agent aligns with the reward function.

Section 20: Baseline Comparison

■ *DISCREPANCY NOTE: The main dossier evaluation metrics and the baseline comparison table show slightly different values for AegisRL. The primary evaluation reports 99.37% attack blocking, 99.79% legitimate access, 0.21% false denial, and +71,363 cumulative reward. The baseline comparison table reports 99.36%, 99.91%, 0.09%, +70,940. These are likely from different evaluation runs. When asked, state the primary evaluation numbers (99.37%, etc.) as the reported results.*

Policy	Attack Block %	Legit Access %	False Deny %	Emerg Deny %	Cumulative R
Random Policy	61.44%	73.61%	26.39%	27.95%	-11,361
Static RBAC	94.54%	75.02%	24.98%	53.56%	+36,480
AegisRL (Trained)	99.36%*	99.91%*	0.09%*	0.14%*	+70,940*

** Values from baseline comparison table in dossier (slightly differ from primary evaluation run)*

Q154. Why does Static RBAC fail so badly on emergency denials (53.56%)?

[Medium]

Viva Answer: Static RBAC checks role and department match. In emergency scenarios, the department match is 0 about 55% of the time – doctors cross department boundaries urgently. Static RBAC sees 'wrong department' and denies. It cannot account for the emergency flag because it has no contextual intelligence. AegisRL learned to prioritize the emergency flag and ALLOW despite dept mismatch.

Q155. Is the baseline comparison fair?

[Hard]

Viva Answer: Mostly, but not perfectly. All three policies are evaluated on the same synthetic scenario distribution, using the same seed=100. The key limitation is that the reward function was designed to favor adaptive contextual decisions, so AegisRL has an inherent advantage. A fairer comparison would use a reward-independent metric like precision/recall on attack detection.

Q156. How does the Random Policy achieve 61.44% attack blocking?

[Hard]

Viva Answer: With 4 actions, random selection picks DENY or CHALLENGE each about 25% of the time. So 50% of random actions are blocking actions. But it's not exactly 61% because RESTRICT also counts for INSIDER_MISUSE. The variance from random sampling pushes it slightly above 50%.

Section 21: Code-Level 'Psycho Sir' Questions

Q157. Show me the exact line where the Q-table is initialized.

[Hard]

Viva Answer: agent.py, line 53: self.q_table = np.zeros((self.num_states, self.num_actions), dtype=np.float64). num_states=1152, num_actions=4. All 4,608 values start at 0.0.

Code Connection: *agent.py line 53*

Q158. Why dtype=np.float64 and not float32?

[Hard]

Viva Answer: float64 gives 15-16 significant decimal digits. The rewards range from -35 to +20 and Q-values accumulate over many updates. float32 (7 digits) could accumulate rounding errors over 35,000 updates per state-action pair. float64 ensures numerical stability.

Q159. Where exactly do you calculate epsilon?

[Hard]

Viva Answer: agent.py, line 99, in decay_epsilon(): self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay). This is called at line 80 of train.py after every episode.

Code Connection: *agent.py line 99; train.py line 80*

Q160. Where exactly is argmax performed?

[Hard]

Viva Answer: agent.py, lines 66-69 in choose_action(). q_row = self.q_table[state_idx] gets the row. max_q = np.max(q_row) finds the maximum. best_actions = np.where(np.isclose(q_row, max_q))[0] finds all actions at the max. np.random.choice(best_actions) picks randomly among them.

Code Connection: *agent.py lines 66-69*

Q161. What happens if I remove max() from the Q-learning update?

[Very Hard]

Viva Answer: The update would use Q(s', a_taken) instead of max Q(s', a'). That turns Q-Learning into SARSA. The agent would learn the value of the behavior policy (epsilon-greedy) rather than the optimal policy. With high epsilon, this would converge to a worse policy.

Code Connection: *agent.py → update(), line 88: np.max(self.q_table[next_state_idx])*

Q162. What happens if you accidentally expose scenario_type to the agent?

[Hard]

Viva Answer: The agent would trivially learn to map scenario→action and achieve perfect performance. But this would be useless in a real system because in production there is no ground-truth scenario label available. The agent must learn from observable features alone.

Q163. What happens if the Q-table has NaN values?

[Hard]

Viva Answer: np.max() on a row with NaN returns NaN. The argmax would be undefined. All subsequent updates involving that state would propagate NaN. The agent would effectively fail silently. The load() function should check for NaN after loading and raise an alert.

Q164. How would you add a fifth action, say MONITOR?

[Hard]

Viva Answer: In config.py: add 'MONITOR' to ACTIONS (now 5 actions), set ACTION_MONITOR=4. In agent.py: num_actions=5 changes q_table shape to (1152, 5). In env.py: add reward cases for MONITOR in calculate_reward(). Delete models/q_table.json and retrain from scratch.

Q165. How would you switch from Q-Learning to SARSA?

[Very Hard]

Viva Answer: In train.py: after env.step(), keep track of the next action chosen by the behavior policy: next_action = agent.choose_action(next_state_idx, training=True). In agent.py update(): instead of np.max(self.q_table[next_state_idx]), use self.q_table[next_state_idx, next_action]. Everything else stays the same.

Q166. How would you convert this to DQN?

[Very Hard]

Viva Answer: Replace the NumPy Q-table with a neural network. Input: the 7 feature values (or one-hot encoded). Output layer: 4 neurons (one Q-value per action). Add a replay buffer to store (s,a,r,s') tuples. Add a target network that's updated every N steps. Training uses mini-batch gradient descent on the Bellman loss. The Q-table file would be replaced by model weights (e.g. .pth for PyTorch).

Section 22: Why Did You Do It This Way?

Q167. Why tabular Q-Learning and not DQN?

[Medium]

Viva Answer: With only 1,152 states, tabular Q-Learning converges exactly to the optimal policy with mathematical guarantees. DQN is an approximation and can diverge or be unstable. For a safety-critical hospital system, exact convergence and full interpretability of every Q-value is much more valuable than the flexibility of neural networks.

Q168. Why offline training and not online learning during deployment?

[Hard]

Viva Answer: In a hospital, you want the access control system to behave predictably and deterministically. Online learning would mean the policy changes as real requests come in – potentially learning from an attacker deliberately crafting requests to manipulate the policy. Offline training on a controlled simulation is safer.

Q169. Why is the Q-table saved as a separate JSON file and not embedded in the code?

[Easy]

Viva Answer: The Q-table is the output of training, not a hardcoded parameter. Separating it means you can retrain and update the model without changing any code. The API loads it at startup, so deploying a new model is just replacing the JSON file.

Q170. Why use FastAPI's /startup event to load the model instead of loading on first request?

[Medium]

Viva Answer: Loading at startup means the first API request doesn't pay the file I/O cost. Also, if the model file is missing, the startup warning is logged immediately rather than silently failing on the first prediction request.

Code Connection: *main.py → startup_event(), line 69*

Section 23: Counterfactual – What If I Change This?

Q171. What if $\alpha = 1$?

[Medium]

Viva Answer: The Q-table would be completely overwritten by each new experience. No memory of past updates. In a deterministic environment this could still converge, but in our stochastic environment the Q-values would oscillate wildly based on the latest reward seen.

Q172. What if $\varepsilon = 0$ from the start?

[Medium]

Viva Answer: The agent would always exploit its initial Q-values (all zeros). With ties, it would pick randomly until some states get updated. But it would never systematically explore all state-action pairs, so many Q-values would remain at zero and the policy would be suboptimal.

Q173. What if emergency DENY penalty is reduced from -35 to -1?

[Hard]

Viva Answer: The agent would no longer be strongly incentivized to avoid denying emergency requests. The emergency denial rate would likely increase significantly because DENY might be chosen more often for suspicious-looking emergency requests (cross-department, off-hours).

Q174. What if ALLOW during STOLEN_CREDENTIAL gets +10 instead of -30?

[Hard]

Viva Answer: The agent would learn that ALLOWing stolen credentials is good. Attack blocking rate would collapse. This shows how critical correct reward engineering is – a wrong reward produces a catastrophically wrong policy.

Q175. What if we remove the Emergency Flag feature?

[Hard]

Viva Answer: The state space shrinks to 576 states (÷2). But the agent can no longer distinguish emergency from non-emergency requests. It would use other features like dept_match and time, but emergency requests that look otherwise suspicious might get denied. Emergency denial rate would rise.

Q176. What if we remove the Location feature?

[Hard]

Viva Answer: The state space shrinks to 384 states (÷3). External attacks would be indistinguishable from internal access by location. The agent would rely more on frequency and time to detect STOLEN_CREDENTIAL. Attack blocking rate for external attacks would likely decrease.

Q177. What if the state space grows to 100,000 states?

[Hard]

Viva Answer: Tabular Q-Learning would become impractical – the Q-table would be 100,000×4 floats = ~3.2 MB, which is still manageable. But visiting all states sufficiently for convergence would

require millions of episodes instead of 35,000. We'd likely switch to DQN.

Q178. What if an attacker knows the state features and tries to manipulate them?

[Very Hard]

Viva Answer: If an attacker knows the frequency threshold is >20/min for BURST_HIGH, they could cap requests at 18/min (ELEVATED) to avoid detection. This is a fundamental limitation – the agent is only as good as the observable features. Countermeasures include randomizing thresholds and adding session-level behavioral features.

Q179. What if the model file is missing when the API starts?

[Medium]

Viva Answer: startup_event() in main.py logs a warning: 'Model file not found. Run train.py to initialize.' The agent object still exists but with an all-zeros Q-table. All requests would get random tie-breaking argmax (always 0=ALLOW from all-zero row). The API wouldn't crash but decisions would be wrong.

Code Connection: *main.py → startup_event(), lines 69-79*

Section 24: RL Theory Beyond the Project

Q180. What is the difference between model-free and model-based RL?

[Medium]

Viva Answer: Model-based RL learns or uses a model of the environment – the transition dynamics P(s'|s,a) and reward function R(s,a). It can plan ahead using the model. Model-free RL (like Q-Learning) directly learns value functions from experience without a model. Model-free is simpler to implement and doesn't need an accurate environment model, but is less sample-efficient.

Q181. What is Dynamic Programming in RL?

[Medium]

Viva Answer: DP uses the known MDP model (transition probabilities and rewards) to compute optimal value functions. Policy Evaluation computes $V_\pi(s)$ for a fixed policy. Policy Improvement finds a better policy. Policy Iteration alternates between them. Value Iteration updates V(s) directly toward the Bellman optimality equation. DP requires knowing P(s'|s,a) exactly.

Q182. Could you solve your environment with Dynamic Programming?

[Hard]

Viva Answer: Technically yes, because our environment has a finite known state space. But we'd need to know the exact transition probabilities between states, which depend on the random sampling in generate_request(). We could estimate these from samples but it's easier to just use Q-Learning directly.

Q183. What is Monte Carlo learning and how does it differ from TD?

[Hard]

Viva Answer: Monte Carlo waits until the end of an episode to update, using the actual total return G_t. TD updates immediately after each step using a bootstrapped estimate. MC has lower bias (uses real returns) but higher variance. TD has lower variance but some bias from bootstrapping. For single-step episodes like ours, MC and TD(0) are equivalent.

Q184. What is on-policy vs off-policy?

[Medium]

Viva Answer: On-policy: the policy used for learning is the same as the policy used for collecting data. SARSA is on-policy. Off-policy: learn about one policy (target policy) while using a different policy (behavior policy) to collect data. Q-Learning is off-policy – it explores with epsilon-greedy but learns about the greedy policy.

Q185. What is importance sampling in off-policy learning?

[Very Hard]

Viva Answer: When the behavior policy π_b and target policy π differ, the expected value under $\pi_b \neq$ expected value under π. Importance sampling reweights the returns by the ratio $\pi(a|s)/\pi_b(a|s)$ to get an unbiased estimate. Q-Learning avoids importance sampling by directly using the max operator instead of following the actual behavior actions.

Q186. What is TD(λ) and how does it relate to TD(0) and Monte Carlo?

[Very Hard]

Viva Answer: TD(λ) is a spectrum. λ=0 gives TD(0) – bootstrapping from the next step. λ=1 gives Monte Carlo – using the full episode return. λ between 0 and 1 interpolates between them using eligibility traces. My project uses TD(0) Q-Learning.

Q187. What is Expected SARSA?

[Very Hard]

Viva Answer: Expected SARSA uses the expected Q-value over all actions weighted by the policy probability, instead of the max (Q-Learning) or the actual next action (SARSA). It reduces variance compared to SARSA and is off-policy. For a deterministic greedy policy it reduces to Q-Learning.

Q188. What is the contraction mapping theorem and why does it matter for Q-Learning?

[Very Hard]

Viva Answer: The Bellman optimality operator is a contraction mapping – applying it repeatedly brings Q-values closer to Q*. Because it's a contraction, iterative application is guaranteed to converge to a unique fixed point Q*. This provides the theoretical convergence guarantee for Q-Learning under the standard assumptions.

Section 25: Connecting Theory to Your Project

Q189. Where exactly is the Bellman equation in your code?

[Medium]

Viva Answer: In agent.py update(), lines 86-92: target = reward + gamma × np.max(self.q_table[next_state_idx]) for non-terminal steps. td_error = target - current_q. self.q_table[state_idx, action] += learning_rate × td_error. That's the exact Bellman-based Q-learning update.

Code Connection: *agent.py → update(), lines 83-93*

Q190. Is your method on-policy or off-policy, and where does that appear in the code?

[Hard]

Viva Answer: Off-policy. In choose_action() (line 57): during training, with probability epsilon a random action is taken. But in update() (line 88): np.max() is used for the target – the greedy optimal action, not the random action that was taken. This decoupling of behavior policy (epsilon-greedy) from target policy (greedy) is the hallmark of Q-Learning's off-policy nature.

Q191. Where is bootstrapping in your code?

[Hard]

Viva Answer: In agent.py update(), line 88: best_next_q = np.max(self.q_table[next_state_idx]). This uses the current Q-table estimate for the next state to compute the update target. We're not waiting for the true return – we're using our own estimate. That's bootstrapping.

Q192. Could you use Monte Carlo for your project?

[Hard]

Viva Answer: Since every episode is one step (done=True), Monte Carlo and TD(0) produce exactly the same update – there's no future to sum up. The total return G_t = r_t is just the immediate reward. So MC and TD are equivalent for my one-step episodic setup.

Q193. What would change if the environment became continuous?

[Hard]

Viva Answer: Tabular Q-Learning would break – the state space would be infinite. We'd need function approximation. The most natural choice is DQN: represent the 7 features as a continuous input vector, use a neural network to map (state) → (Q-values for 4 actions). Training would require replay buffers, target networks, and much more data.

Section 26: Trick Questions

Q194. Is every AI system a Reinforcement Learning system?

[Easy]

Viva Answer: No. Supervised learning trains on labeled examples. Unsupervised learning finds patterns. Only RL involves an agent interacting with an environment and learning from rewards. Most AI systems are supervised classifiers or generative models, not RL agents.

Trap: *Don't say 'all AI is RL' – that's completely wrong.*

Q195. Does Q-Learning need a neural network?

[Easy]

Viva Answer: No. Q-Learning was originally a tabular algorithm. A neural network (DQN) is only needed when the state space is too large to store explicitly. My project uses pure tabular Q-Learning with a NumPy array, no neural networks.

Trap: *Professor might try to get you to conflate Q-Learning with DQN.*

Q196. Does 'model-free' mean there is no model of anything?

[Hard]

Viva Answer: No. 'Model-free' means no learned model of the environment's transition dynamics. The agent still has a model of its Q-values (the Q-table). It doesn't model P(s'|s,a) – that's what 'model-free' means.

Trap: *The word 'model' is overloaded – it means different things in different contexts.*

Q197. Does 'off-policy' mean no policy is being used?

[Hard]

Viva Answer: No. Off-policy means two policies exist: the behavior policy (used for data collection – epsilon-greedy) and the target policy (what we're learning about – greedy). Off-policy means they differ. On-policy means they are the same.

Trap: *'Off-policy' sounds like 'no policy' but it actually means two policies.*

Q198. Does high cumulative reward always mean the system is secure?

[Hard]

Viva Answer: No. Cumulative reward depends entirely on the reward function design. If the reward function is poorly designed, the agent might achieve high reward by gaming the metric. The reward function itself must be validated against real security requirements.

Trap: *This is a key limitation of reward-driven RL – Goodhart's Law.*

Q199. Does 99.37% attack blocking mean the system is secure?

[Hard]

Viva Answer: Not necessarily. It means the agent blocked 99.37% of attacks in the test distribution. Real security depends on whether the test distribution matches actual threats. Novel attacks not in the training/evaluation distribution might bypass the agent. Also, 0.63% unauthorized access in a hospital with millions of records is still significant.

Trap: *Strong evaluation metrics on synthetic data don't guarantee real-world security.*

Q200. Is RESTRICT equivalent to DENY?

[Easy]

Viva Answer: No. DENY returns no data. RESTRICT returns a masked/de-identified view. For insider misuse, RESTRICT is a positive reward (+8) because it contains the breach while providing minimum-necessary access. DENY is also positive (+15) but stronger. They are distinct actions with different risk profiles.

Trap: *Don't say RESTRICT and DENY are the same – they differ in what data is returned.*

Q201. Is an emergency request always legitimate?

[Hard]

Viva Answer: No. In STOLEN_CREDENTIAL scenario generation (env.py line 175), there's a 10% chance is_emergency=1. An attacker might spoof the emergency flag. The agent must learn to look at the entire context, not just the emergency flag.

Code Connection: *env.py line 175: is_emergency = random.choices([0,1], weights=[0.90, 0.10])[0]*

Trap: *Professor expects you to know that emergency flag can be spoofed in stolen credential attacks.*

Q202. Is the Q-table inference latency equal to API latency?

[Hard]

Viva Answer: No. The 0.0184 ms is measured in evaluate.py with perf_counter() around just the argmax call. The actual API POST /predict includes JSON deserialization, Pydantic validation, Supabase write, and network round-trip. Total API latency would be much higher.

Trap: *Never claim 0.0184 ms is the full API response time.*

Q203. Is the database part of the reinforcement learning algorithm?

[Easy]

Viva Answer: No. The database stores the audit trail after decisions are made. The RL decision process (state encoding → Q-table lookup → argmax) happens entirely in memory. The database could be removed without affecting the quality of RL decisions.

Trap: *Professor might try to confuse the architectural components.*

Section 27: Numerical / Calculation Questions

Q204. Calculate state index for: Doctor, dept_match=1, CRITICAL_VIP, OFF_HOURS, EXTERNAL_UNKNOWN, BURST_HIGH, Emergency=1

[Hard]

Viva Answer: Role=0, DeptMatch=1, Sensitivity=3, RequestTime=1, Location=2, Frequency=2, IsEmergency=1. Index = 0×288 + 1×144 + 3×36 + 1×18 + 2×6 + 2×2 + 1 = 0 + 144 + 108 + 18 + 12 + 4 + 1 = 287.

Quick Calculation: *0×288=0, 1×144=144, 3×36=108, 1×18=18, 2×6=12, 2×2=4, 1=1. Sum=287*

Q205. Q-values: ALLOW=5, DENY=2, CHALLENGE=7, RESTRICT=4. Which action does greedy policy select?

[Easy]

Viva Answer: CHALLENGE has the highest Q-value of 7. The greedy policy selects CHALLENGE.

Quick Calculation: *max(5, 2, 7, 4) = 7 → action index 2 → CHALLENGE*

Q206. Q(s,a)=4, reward=10, alpha=0.1, done=True. What is the updated Q-value?

[Medium]

Viva Answer: Since done=True, target = reward = 10. TD_error = 10 - 4 = 6. New Q = 4 + 0.1 × 6 = 4 + 0.6 = 4.6.

Quick Calculation: *New Q(s,a) = 4 + 0.1 × (10 - 4) = 4 + 0.6 = 4.6*

Q207. Q(s,a)=8, reward=12, next Q-values=[10,6,3,9], gamma=0.90, alpha=0.1, done=False. Update Q(s,a).

[Hard]

Viva Answer: max Q(s') = max(10,6,3,9) = 10. Target = 12 + 0.90 × 10 = 12 + 9 = 21. TD error = 21 - 8 = 13. New Q = 8 + 0.1 × 13 = 8 + 1.3 = 9.3.

Quick Calculation: *target=12+0.9×10=21; TD_err=21-8=13; new Q=8+0.1×13=9.3*

Q208. Calculate epsilon after episode 15,000 using decay=0.9998.

[Hard]

Viva Answer: ε = max(0.01, 1.0 × 0.9998^15000). 0.9998^15000 = e^(15000 × ln(0.9998)) ≈ e^(15000 × (-0.0002)) = e^(-3.0) ≈ 0.0498. The dossier reports 0.0498 at episode 15,000 – perfect match.

Quick Calculation: *0.9998^15000 ≈ e^(-3.0) ≈ 0.0498*

Q209. How many Q-table entries are there and what is memory size?

[Easy]

Viva Answer: 1,152 states × 4 actions = 4,608 Q-values. At float64 (8 bytes each): 4,608 × 8 = 36,864 bytes ≈ 36 KB. JSON file is ~81 KB due to ASCII number formatting overhead.

Quick Calculation: *1152 × 4 = 4608 values; 4608 × 8 bytes = 36,864 bytes*

Q210. If evaluation has 5,000 requests with 35% attacks and 0.63% unauthorized access rate, how many breaches occurred?

[Hard]

Viva Answer: Total attacks = 5000 × 0.35 ≈ 1,750 attack requests. Unauthorized access = 0.63% × 1750 ≈ 11 breaches.

Quick Calculation: *1750 × 0.0063 ≈ 11 unauthorized accesses*

Q211. Decode state index 0.

[Medium]

Viva Answer: All features are at their minimum value: Role=0 (DOCTOR), DeptMatch=0 (MISMATCH), Sensitivity=0 (LOW), RequestTime=0 (NORMAL_HOURS), Location=0 (INTERNAL_HOSPITAL), Frequency=0 (NORMAL), IsEmergency=0 (NO). Decode: 0%2=0 (emerg), 0%3=0 (freq), 0%3=0 (loc), 0%2=0 (time), 0%4=0 (sens), 0%2=0 (dept), 0%4=0 (role).

Quick Calculation: *Index 0 → all features = 0: (Doctor, Mismatch, Low, Normal Hours, Internal, Normal, No-Emergency)*

Section 28: Debugging Questions

Q212. Q-values remain zero after 35,000 episodes. Why?

[Hard]

Viva Answer: Possible causes: (1) The agent is not calling update() – check train.py training loop. (2) alpha=0 – check agent constructor. (3) The same state is never visited because the state index is always the same – check state_to_index(). (4) Model was never saved or saved to wrong path – check MODEL_PATH in config.py.

Q213. Agent always chooses ALLOW regardless of input. Why?

[Hard]

Viva Answer: Either: (1) Q-table is all zeros and ALLOW is index 0 – tie-breaking always picks index 0 if np.random.choice is seeded consistently. Or (2) Q-table is loaded incorrectly with all zeros. Check agent.load() return value and verify q_table has non-zero values.

Q214. Epsilon is not decreasing. Why?

[Medium]

Viva Answer: decay_epsilon() is not being called. Check that agent.decay_epsilon() is called inside the training loop after each episode (train.py line 80). Or epsilon_decay might be 1.0 – check the value passed to the agent constructor.

Q215. Index out of bounds error in Q-table. Why?

[Hard]

Viva Answer: state_to_index() returned a value ≥ 1152. This means one of the feature values is out of its expected range. E.g., if role=4 (which doesn't exist), the index calculation would overflow 1151. Check parse_context_to_observation() for improper fallback handling.

Q216. Evaluation gives 100% accuracy. Why is that suspicious?

[Very Hard]

Viva Answer: Perfect scores on test data usually indicate (1) the same seed was used for training and evaluation – data leakage, or (2) the reward function is trivially easy to satisfy, or (3) the test distribution is too narrow. Check that training seed (42) ≠ evaluation seed (100).

Q217. Training reward suddenly becomes NaN. What could cause it?

[Very Hard]

Viva Answer: NaN reward could come from: (1) A divide by zero in reward calculation. (2) NaN propagated from np.max() of a row containing NaN. (3) Integer overflow if feature values are not properly bounded. Check calculate_reward() for any division operations and verify feature bounds.

Q218. API returns 500 for /predict. How do you debug?

[Medium]

Viva Answer: First check FastAPI logs (uvicorn output). The exception message will pinpoint the line. Common causes: (1) Q-table not loaded – agent.q_table is None or all zeros. (2) parse_context_to_observation() threw an exception from unexpected input. (3) Supabase logging

threw an uncaught exception. Add try/except around each component.

Section 29: Security Questions

Q219. Is the system actually secure enough for a real hospital?

[Medium]

Viva Answer: No, not as currently implemented. This is a research prototype. For real deployment, you'd need: real MFA integration for CHALLENGE, actual field-level masking for RESTRICT, JWT authentication on the API, audit log immutability, HIPAA compliance auditing, clinical workflow integration, and formal security testing by a certified healthcare cybersecurity firm.

Q220. Can an attacker exploit the RL model itself?

[Very Hard]

Viva Answer: Potentially yes – this is called adversarial RL. If an attacker can observe which inputs produce which decisions, they could craft requests that land in states where the agent outputs ALLOW. For example, always setting frequency=NORMAL, time=NORMAL_HOURS, location=INTERNAL – even from an external connection – to look like a normal internal request.

Q221. Why should RL not be the only authorization mechanism?

[Hard]

Viva Answer: Defense in depth. RL provides adaptive contextual intelligence but can be manipulated. It should be combined with: strong authentication, network-level firewalls, audit logs with anomaly detection, and clinical workflow-level access controls. RL is one layer, not the entire security architecture.

Q222. What should the fail-safe behavior be if the RL model fails?

[Very Hard]

Viva Answer: Fail-secure: default to DENY or CHALLENGE. Never default to ALLOW. In the current implementation, if the Q-table is all zeros (model not loaded), all Q-values are tied at 0 and ALLOW might be selected by random tie-breaking. This should be fixed to default to CHALLENGE or DENY when no model is loaded.

Code Connection: *main.py → startup_event() should set a 'model_loaded' flag that /predict checks*

Section 30: Limitations

Q223. What are the key limitations of AegisRL?

[Medium]

Viva Answer: Five major ones: (1) Synthetic data – the agent was never trained on real EHR access logs. (2) No generalization – novel attack patterns not in the five scenarios will not be handled well. (3) Discrete state space – nuanced continuous signals like exact request count are binned coarsely. (4) No online learning – the policy doesn't adapt to evolving threats after deployment. (5) Not production-ready – no MFA integration, no real masking, no HIPAA certification.

Q224. What happens to unseen attacks not in the five training scenarios?

[Hard]

Viva Answer: The agent maps them to a state index and applies its learned Q-values. If the observable features of the unseen attack match a state the agent associates with a bad action (e.g., DENY), it will work well by accident. But if the novel attack creates a state pattern not well-visited during training, the agent might make a wrong decision.

Q225. Can the model explain its decisions?

[Hard]

Viva Answer: Yes, in a limited way. explain_decision() in agent.py generates natural language rationale based on the observable features. But this is a hand-crafted explanation, not a true causal explanation. Q-values show what the agent prefers in a state but don't explain why the reward function was designed that way.

Code Connection: *agent.py → explain_decision(), line 109*

Q226. What ethical concerns exist with an RL system making access control decisions?

[Very Hard]

Viva Answer: The reward function encodes ethical choices – what counts as acceptable access. If those choices are biased (e.g., certain roles systematically denied despite legitimacy), the agent perpetuates that bias. Accountability is also unclear – if the agent makes a wrong decision, who is responsible?

Section 31: Future Work

Q227. When would DQN become useful for this project?

[Medium]

Viva Answer: If the feature space is expanded to include continuous signals – exact request timestamps, geographic coordinates, device fingerprints, full query history. These would make tabular Q-Learning infeasible. DQN could handle a continuous state vector and generalize across unseen states through neural network interpolation.

Q228. Could you use multi-agent RL here?

[Very Hard]

Viva Answer: Yes, interestingly. You could model the attacker as a separate adversarial agent trying to maximize breaches, and the access control agent trying to minimize them. This would produce a more realistic adversarial training setup where the agent learns to defend against an adaptive attacker.

Q229. How would you handle concept drift – attackers changing their patterns over time?

[Very Hard]

Viva Answer: Online learning or periodic retraining. The current offline model would become outdated as attack patterns evolve. You'd need a sliding window of recent incidents to retrain the agent or use a continuous learning approach with careful stability mechanisms to prevent catastrophic forgetting.

Q230. How would you add FHIR/HL7 integration for a real hospital deployment?

[Very Hard]

Viva Answer: Parse incoming FHIR R4 access requests, extract the relevant contextual metadata, and map them to the 7 observation features. The RL decision engine would sit as a middleware layer in the FHIR gateway, making decisions before requests reach the EHR system. The audit_logs table would need to be enhanced to store FHIR-compatible audit event resources.

Section 32: Psycho Sir Rapid Fire Round

Answer each question in 1-2 spoken sentences maximum. These may come in rapid succession during viva.

- **What is alpha?**
  The learning rate. Controls how much each experience updates Q-values. In AegisRL: 0.10.
- **Why 0.1?**
  Standard conservative value. Avoids overwriting previous knowledge while still learning meaningfully.
- **What is gamma?**
  The discount factor. How much we value future rewards. In AegisRL: 0.90.
- **Why 0.9?**
  We value near-future rewards highly but not infinitely. 0.9 is standard for episodic RL.
- **What is epsilon?**
  Exploration rate. Probability of taking a random action. Starts at 1.0, decays to 0.01.
- **Why decay epsilon?**
  To transition from exploration early on to exploitation of learned knowledge later.
- **Why 0.9998 decay?**
  Slow enough to explore for most of 35,000 episodes, reaching minimum around episode 23,000.
- **Why 0.01 minimum?**
  Maintains a 1% chance of exploration to avoid prematurely locking in suboptimal policies.
- **Why 35,000 episodes?**
  Enough to visit all 1,152 states multiple times and converge Q-values.
- **Why 1,152 states?**
  4×2×4×2×3×3×2 = 1,152 – product of all feature cardinalities.
- **Why 4 actions?**
  Binary ALLOW/DENY is too coarse. CHALLENGE and RESTRICT handle intermediate cases.
- **Why 7 features?**
  Each feature addresses a distinct attack vector or clinical safety dimension.
- **What is Q-Learning?**
  A model-free, off-policy TD algorithm that learns the optimal action-value function.
- **Is Q-Learning on-policy?**
  No, it's off-policy. It learns the greedy policy while exploring randomly.
- **What is bootstrapping?**
  Using your current estimate of future Q-values to compute the update target, instead of waiting for real return.
- **What is the TD error?**
  The difference between the target (reward + gamma×max Q') and the current Q estimate. delta = target - Q(s,a).
- **Why not SARSA?**

SARSA is on-policy – it learns the epsilon-greedy policy. Q-Learning learns the optimal greedy policy.

- **Why not Monte Carlo?**
  MC waits until episode end. TD updates immediately. For our one-step episodes they're equivalent, but TD is the standard.
- **What is the state space?**
  1,152 discrete states from 7 categorical features.
- **What is the action space?**
  4 actions: ALLOW (0), DENY (1), CHALLENGE (2), RESTRICT (3).
- **What shape is the Q-table?**
  (1152, 4). Verified in agent.py line 53 and models/q_table.json.
- **How many Q-values?**
  4,608 float64 values.
- **What is seed 42?**
  The random seed for the training environment – ensures reproducible training.
- **What is seed 100?**
  The independent random seed for the evaluation environment – ensures test data was never seen in training.
- **What is /predict?**
  The POST endpoint that takes access request context and returns the RL agent's decision.
- **What is /simulate?**
  GET endpoint that generates a synthetic request with ground truth and returns the agent's decision plus reward.
- **What does argmax do?**
  Selects the action index with the highest Q-value in the current state row.
- **What does max() do in update?**
  Finds the best Q-value in the next state – the bootstrapped target for the update.
- **What if alpha = 1?**
  Completely overwrites old Q-values. Too aggressive. Agent won't converge in stochastic environment.
- **What if gamma = 0?**
  Agent only cares about immediate reward. Ignores future consequences.
- **What if epsilon = 0?**
  Pure exploitation from start. Agent never explores. Q-table underexplored. Policy suboptimal.
- **What if reward is wrong?**
  Agent learns the wrong policy. Garbage in, garbage out. Reward engineering is critical.
- **What is your biggest limitation?**
  Synthetic data. The agent was never tested on real clinical access logs under realistic attack conditions.
- **What is your biggest contribution?**
  Modeling hospital access control as an MDP with four nuanced actions and a clinically-aware reward function that achieves 99.37% attack blocking with 0.13% emergency denial rate.
- **Is this actually RL?**

Yes. Agent, environment, state, action, reward, policy, Q-value updates – all present. Classic tabular Q-Learning.

- **Why synthetic attacks?**
  Real EHR attacks involve PHI and cannot be used ethically without proper data use agreements and HIPAA compliance.
- **Why FastAPI?**
  ASGI async framework with automatic Pydantic validation and OpenAPI docs. Faster than Flask.
- **Why Supabase?**
  Hosted PostgreSQL with Python SDK. No server management needed for a prototype.
- **What is ASGI?**
  Asynchronous Server Gateway Interface – the Python standard for async web servers.
- **What is Uvicorn?**
  The ASGI server that runs the FastAPI application.
- **What is an MDP?**
  Markov Decision Process: (S, A, P, R, gamma). Framework for sequential decision-making under uncertainty.
- **What is the Markov property?**
  Future states depend only on the current state, not history.
- **What is V(s)?**
  State-value function – expected total reward from state s following the optimal policy.
- **What is Q(s,a)?**
  Action-value function – expected total reward from state s, taking action a, then following optimal policy.
- **What is pi?**
  Policy – a mapping from states to actions (or probability distributions over actions).
- **What is a return G_t?**
  Sum of discounted future rewards from time t: G_t = r_{t+1} + γ×r_{t+2} + γ²×r_{t+3} + ...
- **What is exploration-exploitation tradeoff?**
  The tension between trying new actions to learn more vs using known good actions to maximize reward.
- **What is the Bellman equation?**
  The recursive relationship: V(s) = max_a [R(s,a) + gamma × sum_s' P(s'|s,a) × V(s')].
- **Where is epsilon decayed?**
  agent.py → decay_epsilon() → line 99. Called in train.py line 80 after every episode.
- **Where is the Q-table saved?**
  agent.py → save() → writes to models/q_table.json as JSON.
- **Where is the Q-table loaded?**
  agent.py → load() → called in main.py startup_event(). Reads q_table.json and converts to NumPy array.
- **What are the 5 scenarios?**
  NORMAL_ACCESS (50%), EMERGENCY_ACCESS (15%), STOLEN_CREDENTIAL (12%), INSIDER_MISUSE (12%), MASS_ACCESS (11%).

- **What is RESTRICT used for?**
  Partial/masked access. Best for insider misuse where you contain the breach without full lockout.
- **What is CHALLENGE used for?**
  Step-up verification. Useful for stolen credentials (attacker can't pass MFA) and normal access with friction.
- **Emergency reward for ALLOW?**
  +20.0
- **Emergency reward for DENY?**
  -35.0 – the maximum penalty.
- **Stolen credential ALLOW reward?**
  -30.0 – direct data breach.
- **Insider misuse RESTRICT reward?**
  +8.0 – containment strategy.
- **Mass access ALLOW reward?**
  -35.0 – catastrophic bulk exfiltration.
- **What is attack blocking rate formula?**
  (Attacks Blocked by DENY or CHALLENGE) / (Total Attack Requests) × 100
- **What is false denial rate formula?**
  (Legitimate Requests Denied) / (Total Legitimate Requests) × 100
- **What is p95 latency?**
  The 95th percentile of all decision latency measurements. 0.0222 ms in our evaluation.
- **Is 0.0184 ms the API latency?**
  No – that's pure RL inference latency. API round-trip is much higher.
- **How many database tables?**
  7: users, patients, medical_records, departments, access_requests, rl_decisions, audit_logs.
- **What is adapt_payload_to_schema?**
  Filters the insert payload to only columns that actually exist in the Supabase table, preventing column errors.
- **What is parse_context_to_observation?**
  Converts incoming API JSON into a discrete Observation namedtuple for the RL agent.
- **What is state_to_index?**
  Maps the 7-tuple Observation to a unique integer 0-1151 using a weighted sum formula.
- **What is index_to_state?**
  Reverses state_to_index using sequential modulo and integer division.
- **What is explain_decision?**
  Generates a natural language rationale for the agent's decision based on observable features.
- **What is the scenario distribution in training?**
  Normal 50%, Emergency 15%, Stolen Credential 12%, Insider Misuse 12%, Mass Access 11%.
- **Why is dept_match always 0 for insider misuse?**
  Insider snooping is by definition accessing records outside your assigned department.

- **Why is frequency always BURST_HIGH for mass access?**
  Mass scraping always produces burst query velocity – it's hardcoded as the defining characteristic.
- **Can training/inference run simultaneously?**
  No. Training runs offline. After training, the model is saved. Inference uses the saved model. They never overlap.
- **How many parameters in the model?**
  4,608 Q-values. No neural network parameters.
- **What is the training speed?**
  ~27,521 episodes per second. Total 35,000 episodes in 1.27 seconds.
- **Why is training so fast?**
  Pure NumPy array operations. No gradient computation. No neural network. One arithmetic formula per episode.
- **What is plotly used for?**
  Interactive training convergence charts (reward + epsilon + TD error) and evaluation action distribution bar charts.
- **What is python-dotenv?**
  Loads SUPABASE_URL and SUPABASE_KEY from .env file into environment variables.
- **What is the CORS policy?**
  allow_origins=['*'] – all origins allowed. Fine for dev, needs restriction in production.
- **What language is the frontend?**
  Vanilla JavaScript (ES6+), Vanilla CSS3, HTML5. No framework.
- **What is the env seed for main.py?**
  seed=2026 for env_simulator used by /simulate endpoint. Different from training (42) and evaluation (100).
- **What is the reward for normal CHALLENGE?**
  +1.0 – small positive reward since legitimate staff can complete MFA with minor friction.
- **What reward for INSIDER CHALLENGE?**
  +6.0 – good because it deters the snooper without full lockout.
- **What reward for MASS ACCESS CHALLENGE?**
  +8.0 – good because automated scripts cannot complete CAPTCHA/MFA.
- **How does index_to_state work?**
  Sequential mod and integer division, peeling off the least significant dimension first (IsEmergency → Frequency → ... → Role).
- **Why np.isclose in tie-breaking?**
  Floating-point equality (==) is unreliable for float64. isclose() handles tiny numerical differences from repeated Q-updates.
- **Is the Q-table updated during /predict?**
  No. choose_action(training=False) is read-only. update() is never called during API requests.
- **What is the hospital seed for evaluation?**
  seed=100 passed to AccessRequestEnvironment in evaluate_agent().
- **What is baseline cumulative reward?**

Random: -11,361. Static RBAC: +36,480. AegisRL: +70,940 (baseline table) or +71,363 (primary eval).

Section 33: Professor Follow-Up Chains

Each chain simulates how a professor will drill deeper. Read these as a conversation.

Chain 1: Why Q-Learning?

Q: Why did you use Q-Learning?

A: Because we have a finite discrete state space of 1,152 states where tabular Q-Learning can provably converge to the optimal policy without approximation.

→ **Q: Why is Q-Learning suitable for your state space?**

A: 1,152 states is small enough to store exact Q-values. Every state-action pair can be visited multiple times in 35,000 episodes, guaranteeing sufficient coverage for convergence.

→ → **Q: Why not DQN?**

A: DQN is for large or continuous state spaces. For 1,152 states, a neural network adds complexity, approximation error, and instability with no benefit. Tabular Q-Learning is the exact right tool here.

→ → → **Q: What would change if your state space became continuous?**

A: We'd need DQN or another function approximator. The input would be a vector of 7 feature values (or one-hot encoded), the output would be 4 Q-values, and we'd need replay memory and a target network.

→ → → → **Q: Then why didn't you use DQN from the beginning?**

A: Because my current state space doesn't need it. DQN is a solution to a problem I don't have. Using it would be overengineering a simple problem and losing convergence guarantees.

Chain 2: Reward Function Design

Q: Why did you design the reward function manually?

A: Because domain knowledge about patient safety vs data security is well-established and can be quantified. A doctor denied during emergency is life-threatening. A stolen credential breach is a legal violation. We can assign relative penalties.

→ **Q: Isn't the agent just learning the rules you encoded?**

A: The reward encodes what outcomes are good or bad. But which combination of observable features produces which outcome – that's learned by the agent. The agent discovers that external+off-hours+high-sensitivity triggers DENY without me writing an explicit rule for that combination.

→ → **Q: What if your reward values are wrong?**

A: The agent would learn the wrong policy. If emergency DENY was rewarded +5 instead of penalized -35, the agent would learn to deny emergencies. Reward engineering is the most critical design decision in RL.

→ → → **Q: How would you validate the reward function in a real setting?**

A: By consulting clinical staff (what makes a good access decision), legal/compliance teams (HIPAA minimum-necessary principles), and cybersecurity experts (threat models). Then testing the emergent policy against diverse scenarios.

Chain 3: Emergency Access

Q: Why does EMERGENCY_ACCESS DENY get -35?

A: Because denying a doctor access to patient vitals during a cardiac arrest can cause patient death. That's the worst possible outcome in a hospital system.

→ Q: But can't an attacker just set emergency_status=True?

A: Yes, and this is a known risk. In env.py's STOLEN_CREDENTIAL scenario, 10% of requests have is_emergency=1 – simulating exactly that. But the agent also looks at location, frequency, and dept_match to make a holistic decision.

→ → Q: So if emergency is True but location is EXTERNAL_UNKNOWN, what does the agent do?

A: It depends on what Q-value the agent has learned for that specific state. In practice, external emergency access is very rare and suspicious. The agent likely has a moderate Q-value for CHALLENGE in that state – asking for verification without outright denying.

→ → → Q: What if the agent DOES deny an emergency from external?

A: It's a 'false denial' from the safety perspective, but potentially the right security call. This is the fundamental tradeoff the reward function tries to balance. The -35 penalty ensures the agent is very conservative about emergency denials.

Chain 4: State Space Design

Q: Why these specific 7 features?

A: Each feature addresses a documented threat vector. Location catches remote attacks. Frequency catches scraping. Emergency flag catches the false-denial problem. Together they give the agent enough context to distinguish 5 attack scenarios from legitimate access.

→ Q: Why discretize frequency into only 3 buckets?

A: Clinical literature suggests ≤5/min is routine, 6-20/min is slightly elevated, and >20/min is anomalous scraping. Three meaningful clinical boundaries. Adding more buckets would grow the state space without adding clinical insight.

→ → Q: What happens if an attacker stays at exactly 20/min?

A: That maps to ELEVATED (6-20/min bucket). The agent's response to ELEVATED frequency depends on other features. A doctor at ELEVATED from internal hospital is probably fine. The same from EXTERNAL_UNKNOWN would still get flagged by the location feature.

→ → → Q: Why only 4 roles? What about pharmacist, lab technician?

A: For this prototype, the 4 roles (DOCTOR, NURSE, STAFF, RESEARCHER) represent the primary clinician archetypes. Adding more roles would expand the state space. This is explicitly listed as future work.

Chain 5: Convergence

Q: How do you know the agent converged?

A: The TD error decreased from 11.26 at episode 1000 to 0.455 at episode 35000, and the average reward plateaued at 14.07. These are the standard empirical convergence signals for tabular Q-Learning.

→ **Q: Does decreasing TD error prove convergence?**

A: It's strong evidence but not formal proof. Formal proof would require showing Q-values stopped changing and match the theoretical optimal. With constant alpha=0.1, there's always some residual fluctuation.

→ → **Q: What would you do to prove convergence more rigorously?**

A: Monitor Q-value changes between checkpoints – if max|Q_new - Q_old| < threshold, declare convergence. Alternatively, run multiple random seeds and report variance in final performance.

→ → → **Q: Why does average reward not reach the maximum possible?**

A: Because epsilon=0.01 means 1% of actions are still random during training. Random actions generate suboptimal rewards that drag down the average. At evaluation (epsilon=0), the reward is 99% of maximum.

Chain 6: MDP Formulation

Q: Is this really an MDP?

A: Yes – it has states (7-tuple), actions (4 decisions), transition dynamics (random request sampling), reward function (piecewise reward matrix), and a discount factor (0.90).

→ **Q: Is your state fully Markov?**

A: Within the discrete abstraction, yes. The history-dependence of frequency is captured by pre-computing the frequency bucket before the decision. But strictly, if future requests depend on the history of past decisions, the full process isn't Markov at the raw data level.

→ → **Q: What information is missing from your state?**

A: History of past access decisions for the same user. Device fingerprint. Geographic coordinates. These could improve detection but would expand the state space significantly.

→ → → **Q: Is the transition model deterministic?**

A: No. The next state is randomly sampled by generate_request() according to scenario probabilities. The transition is stochastic.

Chain 7: Alpha and Learning Rate

Q: Why alpha = 0.1?

A: It's a conservative, well-tested learning rate. 10% update per step means the agent doesn't overreact to individual stochastic rewards but still learns at a reasonable pace.

→ **Q: What's wrong with alpha = 0.5?**

A: With stochastic rewards, a large alpha would cause Q-values to oscillate. If one episode gives reward -35 and the next gives +20, the Q-value would swing dramatically. Alpha = 0.1 smooths this out.

→ → **Q: What's the mathematical condition for alpha to guarantee convergence?**

A: The Robbins-Monro conditions: sum(alpha_t) = infinity and sum(alpha_t^2) < infinity. A decaying learning rate like alpha_t = 1/t satisfies this. A constant alpha=0.1 violates the second condition but works well empirically.

→ → → **Q: So tabular Q-Learning with constant alpha doesn't formally converge?**

A: With constant alpha it converges to a neighborhood around the optimal, not exactly to Q*. The Q-values fluctuate around the true values. For practical purposes, the fluctuation is negligible if alpha is small enough.

Chain 8: Stolen Credential vs Insider Misuse

Q: What's the difference between STOLEN_CREDENTIAL and INSIDER_MISUSE?

A: Stolen credential: an outsider using a valid employee's credentials, typically from external networks, off-hours, high-value targets. Insider misuse: an actual employee snooping on records outside their department, from internal network, during working hours.

→ **Q: So both have dept_match = 0?**

A: Yes. Stolen credential has 80% probability of dept_match=0, and insider misuse always has dept_match=0. But they differ in location – insider is 90% internal, stolen credential is 85% external.

→ → **Q: How does the agent distinguish them?**

A: Primarily by location. Internal + dept_match=0 + CRITICAL_VIP → likely insider misuse → RESTRICT or DENY. External + dept_match=0 + OFF_HOURS → likely stolen credential → DENY or CHALLENGE.

→ → → **Q: What if a stolen credential is used from inside the hospital?**

A: That's the 15% case in stolen credential generation (VPN_OFFSITE) or the even rarer internal case. If it looks exactly like insider misuse, the agent would treat it as such – likely RESTRICT or DENY, which is still the right call for unauthorized access.

Chain 9: Database Architecture

Q: Why does the database have 7 tables?

A: Each table represents a distinct entity or event type. users and patients are entities. medical_records are the resource being accessed. departments organize the hospital. access_requests, rl_decisions, and audit_logs capture the audit trail at different granularities.

→ **Q: Why not combine access_requests and rl_decisions into one table?**

A: Separation of concerns. A request might be made before the RL decision is computed. The RL decision references the request via foreign key. This allows independent logging and querying.

→ → **Q: What happens if the foreign key constraint fails?**

A: db.py's resolve_entity_id() tries multiple resolution strategies: integer lookup, code-column lookup, digit extraction. If all fail, it uses get_fallback_id() which returns any valid existing ID from that table. If even that fails, it returns None and the insertion is skipped gracefully.

→ → → Q: Why is reward stored as q_values.get(action, 0) instead of the actual reward?

A: Because actual rewards require hidden_meta (the scenario ground truth), which is not available during real API inference. The Q-value of the selected action is the closest proxy available at inference time.

Chain 10: Evaluation Methodology

Q: Why is the evaluation done on 5,000 requests?

A: The scenario distribution weights [0.50, 0.15, 0.12, 0.12, 0.11] mean the smallest scenario group gets about 550 samples – enough for statistically meaningful rates with low variance.

→ Q: Is there data leakage in your evaluation?

A: No. Training uses env seed=42, evaluation uses env seed=100. Different seeds generate completely independent request sequences. The Q-table was built without seeing any of the evaluation requests.

→ → Q: What does 99.37% attack blocking actually mean?

A: 99.37% of attack-type requests (stolen credential + insider misuse + mass access) were blocked by DENY or CHALLENGE. The denominator is only attack requests, not all 5,000.

→ → → Q: Why is the legitimate access rate formula counting CHALLENGE and RESTRICT as 'granted'?

A: Because in evaluate.py lines 117-118, legitimate_granted counts ALLOW, CHALLENGE, or RESTRICT. The rationale is that CHALLENGE gives eventual access after verification, and RESTRICT gives partial access. Only DENY is counted as a failure for legitimate users.

Section 34: Code Walkthrough Viva

Q231. Walk me through TabularQLearningAgent.__init__(). (agent.py line 34)

[Medium]

Viva Answer: Inputs: learning_rate=0.1, discount_factor=0.90, epsilon_start=1.0, epsilon_min=0.01, epsilon_decay=0.9997 (default; train.py passes 0.9998 explicitly). It stores all hyperparameters as instance variables. Key line 53: self.q_table = np.zeros((1152, 4), dtype=np.float64) – all Q-values start at 0. If removed: agent has no Q-table. All predictions fail.

Code Connection: *agent.py → __init__(), lines 34-55*

Q232. Walk me through AccessRequestEnvironment.state_to_index(). (env.py line 82)

[Hard]

Viva Answer: Input: an Observation namedtuple with 7 integer fields. Algorithm: weighted sum – each feature multiplied by the product of cardinalities of all features after it. Output: integer in [0, 1151]. Time complexity: O(1) – 7 multiplications and 6 additions. Why needed: NumPy array indexing requires integer, not tuple. Possible bug: if any feature value exceeds its cardinality (e.g., role=5), the index overflows 1151 → array out of bounds.

Code Connection: *env.py → state_to_index(), lines 81-95*

Q233. Walk me through calculate_reward(). (env.py line 266)

[Hard]

Viva Answer: Inputs: obs (Observation), action (int 0-3), meta (dict with scenario and is_legitimate). Algorithm: nested if-elif checks meta['scenario'], then nested if-elif checks action. Returns (reward, outcome_info). Why needed: provides training signal. Without it, agent has no feedback and cannot learn. Possible bug: if a new scenario is added but not handled, reward=0.0 by default (line 401) – silent wrong reward.

Code Connection: *env.py → calculate_reward(), lines 266-403*

Q234. Walk me through evaluate_agent(). (evaluate.py line 30)

[Hard]

Viva Answer: Inputs: model_path, num_samples=5000, seed=100. Loads agent, creates fresh environment, loops num_samples times. For each: generate_request(), state_to_index(), choose_action(training=False), calculate_reward(). Increments counters for attacks_blocked, unauthorized_allowed, legitimate_granted, etc. After loop: computes rates, saves JSON, generates Plotly chart. Why needed: provides unbiased performance estimate on unseen data. Why seed different from training: prevents data leakage.

Code Connection: *evaluate.py → evaluate_agent(), lines 30-194*

Q235. Walk me through log_access_and_decision(). (db.py line 337)

[Hard]

Viva Answer: Inputs: all request fields, action, q_values, latency, state_index, explanation. Resolves foreign keys for user_id, patient_id, record_id via resolve_entity_id(). Inserts into access_requests, gets back request_id. Inserts into rl_decisions linking to request_id. Inserts into audit_logs. Returns True/False. Why needed: immutable audit trail for compliance. Possible bug: if

resolve_entity_id() returns None for any ID, insertion is skipped silently.

Code Connection: *db.py → log_access_and_decision(), lines 337-409*

Section 35: Show Me On The Project

- **Show me where the Q-table is initialized.**

```
agent.py, line 53: `self.q_table = np.zeros((self.num_states, self.num_actions), dtype=np.float64)`
```

- **Show me where the state is encoded.**

```
env.py, lines 86-95: state_to_index() – the weighted sum formula
```

- **Show me where the reward is calculated.**

```
env.py, lines 266-403: calculate_reward() – the full piecewise if-elif tree
```

- **Show me where epsilon is decayed.**

```
agent.py, line 99: `self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)`
```

- **Show me where action selection occurs.**

```
agent.py, lines 62-69: choose_action() – the epsilon-greedy branch with tie-breaking
```

- **Show me where the Bellman update is computed.**

```
agent.py, lines 83-93: update() – target computation, td_error, Q-table update
```

- **Show me where the API endpoint is defined.**

```
main.py, line 207: `@app.post('/predict', response_model=DecisionResponseSchema)`
```

- **Show me where database insertion occurs.**

```
db.py, line 338: log_access_and_decision() function
```

- **Show me where the model is loaded.**

```
main.py, lines 69-79: startup_event() → agent.load(MODEL_PATH)
```

- **Show me where evaluation metrics are computed.**

```
evaluate.py, lines 128-134: attack_blocking_rate, legitimate_access_rate, etc.
```

- **Show me where scenarios are generated.**

```
env.py, lines 118-222: generate_request() with scenario_override
parameter
```

- **Show me where hidden labels are separated from observations.**

```
env.py, lines 203-220: generate_request() returns (obs, hidden_meta)
separately – obs is the Observation namedtuple, hidden_meta contains
scenario and is_legitimate
```

- **Show me where the frontend calls the backend.**

```
static/js/access.js: fetch('/predict', {method: 'POST', body:
JSON.stringify(requestData)})
```

- **Show me where tie-breaking in argmax is handled.**

```
agent.py, lines 67-69: np.where(np.isclose(q_row, max_q))[0] then
np.random.choice(best_actions)
```

Section 36: Final Viva Self-Test

Test yourself without looking at the answers. Cover the answer key below.

ST1. What is the full Q-table shape and why?

ST2. What is the exact formula for state_to_index?

ST3. What is the epsilon after 15,000 episodes?

ST4. What reward does DENY get during EMERGENCY_ACCESS?

ST5. What reward does RESTRICT get during INSIDER_MISUSE?

ST6. What reward does ALLOW get during MASS_ACCESS?

ST7. What is the difference between CHALLENGE and ALLOW?

ST8. Why is Q-Learning off-policy?

ST9. What is bootstrapping in Q-Learning?

ST10. Why is done=True in every episode of AegisRL?

ST11. What happens if alpha = 0?

ST12. How is the TD error calculated in agent.py? (Write the formula)

ST13. Where exactly does epsilon decay happen?

ST14. What are the 7 observation features?

ST15. What makes INSIDER_MISUSE detectable in the feature space?

ST16. How does STOLEN_CREDENTIAL differ from INSIDER_MISUSE in features?

ST17. What is P(s'|s,a) in your specific environment?

ST18. Why is the evaluation seed 100 and not 42?

ST19. Is the decision latency 0.0184 ms the API round-trip? Why/why not?

ST20. What does attack_blocking_rate include in its denominator?

ST21. Why does Static RBAC have 53.56% emergency denial rate?

ST22. What is the role of python-dotenv in config.py?

ST23. What is ASGI?

ST24. What is adapt_payload_to_schema() and why is it needed?

ST25. What happens if the Q-table JSON file is missing at startup?

ST26. Why are there two tables – access_requests and rl_decisions – instead of one?

ST27. What is the training speed in episodes per second?

ST28. How would you convert Q-Learning to SARSA? (One-line change)

ST29. What is the multiplier 288 in state_to_index and where does it come from?

ST30. Name one critical limitation of AegisRL and one concrete future improvement.

ST31. Can an attacker bypass the RL decision by calling /predict directly?

ST32. What does explain_decision() use to generate its rationale?

ST33. What is the cumulative reward across 5,000 evaluation requests?

ST34. What is the p95 latency and what does p95 mean?

ST35. How does the environment generate the next state after an action?

ST36. What is the default role if an invalid role string is sent to /predict?

ST37. Where is np.isclose used and why is it needed instead of ==?

ST38. What is the training environment seed?

ST39. What is the minimum epsilon and why not 0?

ST40. What are the five dashboard views in the frontend SPA?

ST41. Is the Q-table recomputed or loaded fresh per API request?

ST42. What is the Markov property and is your environment fully Markov?

ST43. What would change if you added a new feature 'Device Type' with 3 values?

ST44. What is resolve_entity_id() doing in db.py?

ST45. Why does db.py use two schema inspection strategies (OpenAPI + sample query)?

ST46. What does the training checkpoint at episode 5,000 show?

ST47. How many non-zero Q-table entries are expected after training?

ST48. What is the actual epsilon_decay used during the documented training?

ST49. Why is the mass access scenario always BURST_HIGH frequency?

ST50. What is the research gap that AegisRL addresses?

SELF-TEST ANSWER KEY

ST1 Answer: (1152, 4) – 1152 states × 4 actions. 4,608 float64 values.

ST2 Answer: role×288 + dept_match×144 + sensitivity×36 + time×18 + location×6 + frequency×2 + is_emergency

ST3 Answer: 0.9998^15000 ≈ e^(-3.0) ≈ 0.0498 (matches dossier checkpoint)

ST4 Answer: -35.0 – the maximum negative reward (denying a life-saving emergency)

ST5 Answer: +8.0 – partial masked access is a valid containment strategy

ST6 Answer: -35.0 – catastrophic mass exfiltration

ST7 Answer: CHALLENGE requires step-up MFA verification before granting access. ALLOW grants immediately.

ST8 Answer: The update uses max Q(s',a') – the greedy optimal action – not the epsilon-greedy action actually taken during training.

ST9 Answer: Using current Q-value estimates to compute the update target instead of waiting for the true return.

ST10 Answer: Because each access request is an independent decision. done=True is set in env.step() line 252.

ST11 Answer: Q-table never updates – agent learns nothing.

ST12 Answer: td_error = target - current_q; target = reward (if done=True)

ST13 Answer: agent.py line 99 in decay_epsilon(): self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay). Called from train.py line 80.

ST14 Answer: Role, DeptMatch, Sensitivity, RequestTime, Location, Frequency, IsEmergency

ST15 Answer: dept_match=0 always, CRITICAL_VIP sensitivity (75%), internal location (90%), no burst frequency

ST16 Answer: Stolen: external location (85%), off-hours (80%), no dept constraint. Insider: internal (90%), dept_match=0 always, during work hours (60%).

ST17 Answer: The probability of transitioning to next state s' given (s,a). In our case: the scenario distribution weights in generate_request() – independent of s and a.

ST18 Answer: To ensure test data is independent of training data. Same seed = data leakage.

ST19 Answer: No. 0.0184 ms is pure in-memory argmax latency. API round-trip includes JSON parsing, Pydantic validation, Supabase write, and network.

ST20 Answer: Total attack-type requests (STOLEN_CREDENTIAL + INSIDER_MISUSE + MASS_ACCESS), NOT all 5,000.

ST21 Answer: Static RBAC denies cross-department access. Emergencies often have dept_match=0 (55% probability in env.py). RBAC has no emergency awareness.

ST22 Answer: Loads SUPABASE_URL and SUPABASE_KEY from .env file into environment variables via load_dotenv().

ST23 Answer: Asynchronous Server Gateway Interface – the Python standard for running async web frameworks like FastAPI.

ST24 Answer: Filters an insert payload to only include columns that actually exist in the Supabase table, preventing PGRST204 column-not-found errors.

ST25 Answer: startup_event() logs a warning. agent.q_table remains all-zeros. Inference continues but all Q-values are 0 → random tie-breaking always picks ALLOW (index 0).

ST26 Answer: Separation of concerns. access_requests logs what was requested; rl_decisions logs what the agent decided. Different access patterns and retention policies.

ST27 Answer: ~27,521 episodes/second. Total time: 1.27 seconds.

ST28 Answer: In update(), change np.max(self.q_table[next_state_idx]) to self.q_table[next_state_idx, next_action] where next_action is the actual next action chosen by epsilon-greedy.

ST29 Answer: 288 = 2×4×2×3×3×2 – the product of cardinalities of all features after Role. Each unit of Role spans 288 state-action sub-combinations.

ST30 Answer: Limitation: synthetic data. Improvement: train on real anonymized hospital access logs under proper HIPAA data use agreements.

ST31 Answer: Yes. The API has no authentication. Anyone who can reach port 8000 can POST to /predict.

ST32 Answer: The observable features of the Observation namedtuple: location, frequency, dept_match, sensitivity, role, emergency flag.

ST33 Answer: +71,363.0 (primary evaluation run value in AegisRL_Project_Facts.md)

ST34 Answer: p95 = 95th percentile = 0.0222 ms. It means 95% of decisions are faster than 0.0222 ms. Computed with np.percentile(decision_times_ms, 95).

ST35 Answer: env.step() calls generate_request() at line 253 to sample a fresh independent request. The next state is not caused by the action – each episode is independent.

ST36 Answer: DOCTOR (fallback in ROLE_MAP.get(raw_role, ROLE_MAP['DOCTOR']) in env.py line 413).

ST37 Answer: Because multiple Q-updates can leave Q-values very close but not exactly equal due to floating-point precision. np.isclose() handles those cases correctly.

ST38 Answer: 42 – passed to AccessRequestEnvironment(seed=42) in train.py line 47.

ST39 Answer: 0.01. Not 0 to maintain a 1% exploration floor, preventing premature convergence to a suboptimal policy.

ST40 Answer: Hospital Staff Dashboard, Patient Health Records, RL Decision Engine, Security Administrator Dashboard, Cyber Attack Simulator, Access History & Audit Trail.

ST41 Answer: Loaded once at startup in startup_event(). Never recomputed per request.

ST42 Answer: Future depends only on current state, not history. Mostly Markov; frequency as a pre-computed category is an approximation.

ST43 Answer: State space grows by ×3: 1,152 × 3 = 3,456 states. Q-table becomes (3456, 4). Retrain needed. Update TOTAL_STATES in env.py and all index multipliers.

ST44 Answer: Safely resolves a string or int identifier to an existing foreign key integer in Supabase, handling string codes, digit extraction, and fallback to any valid existing ID.

ST45 Answer: OpenAPI introspection might be unavailable; sample query verification catches columns not in OpenAPI spec. Belt-and-suspenders approach.

ST46 Answer: Avg reward=+7.10, epsilon=0.3678, TD error=5.1999. Agent has begun learning but still 37% exploration.

ST47 Answer: The dossier mentions 1,276+ non-zero entries (from FINAL_TEST_REPORT.md). Not all 4,608 are non-zero because some states may not have been visited during training.

ST48 Answer: 0.9998 – train.py explicitly passes epsilon_decay=0.9998 to train_agent(), overriding the agent.py constructor default of 0.9997.

ST49 Answer: Because burst frequency >20 queries/min is the defining characteristic of automated data scraping. Hardcoded in env.py line 196.

ST50 Answer: Existing systems treat authorization as binary (ALLOW/DENY). AegisRL introduces non-binary intermediate actions (CHALLENGE, RESTRICT) in a unified RL framework with a clinically-aware reward that balances patient safety against data security.

Good Luck in Your Viva!

Every answer in this document is grounded in actual project code. If a professor asks something not covered here, fall back to first principles: state the observable fact from the code, explain the design reasoning, and flag any honest limitations.

AegisRL · Q-Learning · FastAPI · Supabase · 1,152 States · 4 Actions · 35,000 Episodes